Lecture notes on pretraining

Lecture by

David Louapre

Hugging Face

MVA

Fundamentals of Large Language Models

Lecture 3 • 2026

Unofficial course notes

Preface

Pretraining large language models requires allocating finite data, memory, compute, and time. Controlled experiments with small chess models establish the optimization and hyperparameter-transfer principles used to study scaling laws.

These lecture notes cover pretraining in David Louapre's 2026 MVA course on large language models. They develop the course material through mathematical derivations, experimental analysis, and references to the research literature. The two slide decks contain 90 and 33 pages. References of the form **P1:25** mean page 25 of the first PDF, counted from the title page; **P2:4** means page 4 of the second.

The exposition combines experimental results, mathematical derivations, and context from primary sources. Terminology and numerical claims are checked against slide equations, plots, and the cited research. Uncertain or conflicting claims are identified explicitly. Appendix A collects substantive clarifications and figure provenance.

Figures retain the data and annotations of the original slides. Duplicate appearances are consolidated; paper title-page screenshots and code screenshots are replaced by bibliographic entries and explanations. Plot readings are approximate unless a value is printed explicitly. No experiment in the slides is claimed to have been rerun for these notes. Added derivations and qualifications are independent explanations.

The exercises provide reproducible experimental protocols and the reported results. Their quoted durations depend on the original hardware and implementation. Research references were checked against primary sources available on 6 November 2026.

Notation and conventions

Symbol	Meaning and usual unit
N	Parameter count; active, stored, or non-embedding counts are distinguished where needed.
d, L	Transformer width and number of layers.
H, d_h	Number of attention heads and dimension per head; a is also used for head dimension in Chapter 1.
V	Vocabulary size.
s, T	Sequence length or number of positions in a sequence.
D	Training-token horizon; real tokens and processed padded positions are distinguished.
G	Number of chess games processed.
B	Optimizer batch size, in sequences or tokens as specified; b often denotes a microbatch.
C	Training compute, in FLOPs.
P, u	Peak hardware FLOP/s and model FLOPs utilization (MFU).
θ, g	Model parameters and their loss gradient.
η, λ	Learning rate and the AdamW decay coefficient.
β_1, β_2	Adam first- and second-moment averaging coefficients.
$\mathcal{L}$	Mean next-token cross-entropy, using natural logs unless specified.
E, A, B, α, β	Local coefficients in a fitted scaling-law loss surface; here B is a coefficient, not batch size.

Decimal prefixes are used for model and token counts: K denotes 10^3, M denotes 10^6, B denotes 10^9, and T denotes 10^{12}. Context lengths quoted as 8K or 64K follow the source's conventional labels and may denote powers of two in the implementation. Memory units depend on the profiler; MB and MiB are not interchangeable.

FLOPs counts operations; FLOP/s measures a rate. A multiplication plus an addition counts as two FLOPs in the matrix-multiplication estimates used here. Plot labels and fitted coefficients retain the source's units. Numerical recommendations from the chess experiments are specific to that setting.

Contents

Preface **i**

Notation and conventions **ii**

1 The pretraining problem and the chess laboratory **1**
1.1 What pretraining produces . . . 1
1.2 Resources and choices form a coupled problem . . . 1
1.3 Anatomy and approximate parameter count . . . 2
1.4 Why chess is a useful laboratory . . . 3
1.5 Three token counts that must not be confused . . . 4

2 Accounting for compute, memory, and throughput **6**
2.1 Deriving the six-N-D rule . . . 6
2.2 From FLOPs to wall-clock time . . . 7
2.3 Arithmetic intensity and the roofline picture . . . 7
2.4 Where the memory goes . . . 7
2.5 Exercise 1: measuring the device, not guessing . . . 9
2.6 Microbatch, accumulation, and global batch . . . 9
2.7 Width changes utilization . . . 10
2.8 Reading power laws . . . 10

3 Optimization: AdamW and its timescales **12**
3.1 From stochastic gradient descent to Adam . . . 12
3.2 Why weight decay must be decoupled . . . 12
3.3 Interpreting the beta parameters . . . 13
3.4 Weight decay when training for roughly one pass . . . 14
3.5 The training loop and faithful restarts . . . 14

4 Learning-rate schedules and reusable runs **16**
4.1 Why warm up and why cool down? . . . 16
4.2 Cosine with warmup . . . 16
4.3 Warmup–stable–decay (WSD) . . . 17
4.4 Exercise 2: what the curves actually establish . . . 17
4.5 Branching turns one long trajectory into several experiments . . . 17
4.6 Alternatives and the scope of the result . . . 18

5 Learning rate, batch size, and training horizon 20
5.1 Exercise 3a: find the useful learning-rate range . . . 20
5.2 How much should we trust a single run? . . . 20
5.3 Batch size: hardware efficiency versus statistical efficiency . . . 21
5.4 Exercise 3c: tune batch size and learning rate jointly . . . 22
5.5 The optimal learning rate depends on the horizon . . . 23
5.6 Connecting the small experiment to large-scale practice . . . 26

6 Transferring a recipe to wider models 27
6.1 The empirical problem with standard parameterization . . . 27
6.2 Why width changes the meaning of a parameter update . . . 27
6.3 What maximal update parameterization proposes . . . 28
6.4 How to validate transfer experimentally . . . 28

7 Compute-optimal scaling and a prediction tested 30
7.1 What should we do with more compute? . . . 30
7.2 Chinchilla's approximately balanced rule . . . 30
7.3 IsoFLOP curves and a loss surface . . . 31
7.4 Why careful tuning changes a scaling law . . . 34
7.5 Training-optimal is not lifetime-cost-optimal . . . 34
7.6 The chess scaling experiment . . . 35

8 Selecting and preparing pretraining data 40
8.1 The objective changes when the dataset changes . . . 40
8.2 From a web crawl to a training corpus . . . 40
8.3 Content filtering and text extraction . . . 41
8.4 Deduplication: exact matches and near matches . . . 42
8.5 Personal data and benchmark contamination . . . 43
8.6 Model-based filtering and educational quality . . . 44
8.7 Rephrasing and synthetic pretraining text . . . 44
8.8 Mixtures, midtraining, and context extension . . . 45

9 Monitoring a run and investigating instability 48
9.1 Read several signals together . . . 48
9.2 A rising gradient norm need not mean divergence . . . 49
9.3 OLMo: stability involves data, architecture, and optimization . . . 49
9.4 Marin: a recoverable spike and a damaged trajectory . . . 49
9.5 A disciplined diagnostic procedure . . . 52

10 Numerical precision, memory, and parallel training 53
10.1 Floating-point formats: range and precision . . . 53
10.2 Mixed precision and why master weights matter . . . 54
10.3 FP8: scale each tensor appropriately . . . 55
10.4 Activation recomputation . . . 55

10.5 Three basic forms of parallelism 56
10.6 Mistral Large 4: a closing synthesis 57

11 Reproducing the experimental logic 58
11.1 The complete sequence 58
11.2 What to save for every run 58
11.3 Worked checks 59
11.4 Questions for self-assessment 60

A Editorial clarifications and figure provenance 61
A.1 Corrections that affect interpretation 61
A.2 Figure provenance and consolidation 62

B A guide to the research and technical references 63
B.1 Optimization and hyperparameter transfer 63
B.2 Scaling and large-run recipes 64
B.3 Data sources, curation, and mixtures 65
B.4 Numerics, memory, and systems 66

Bibliography 67

Chapter 1

The pretraining problem and the chess laboratory

Lecture slides: P1:1–15.

1.1 What pretraining produces

Pretraining learns a probability distribution over token sequences. For a sequence $x_{1:T}$, an autoregressive model factorizes the joint probability as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}). \tag{1.1}$$

The usual objective is the mean negative log-likelihood over valid prediction targets:

$$\mathcal{L}(\theta) = -\frac{1}{M} \sum_{i,t} m_{i,t} \log p_\theta(x_{i,t} \mid x_{i,<t}), \qquad M = \sum_{i,t} m_{i,t}, \tag{1.2}$$

where $m_{i,t} = 0$ for ignored positions, such as padding, and 1 for valid targets. With natural logarithms, loss is measured in nats per token and perplexity is $\exp(\mathcal{L})$. A uniform predictor over V tokens has loss $\log V$; for a 67-token vocabulary this is about 4.20.

The result is a *base model*: a model trained to continue the training distribution. Following instructions, choosing a conversational style, and optimizing preferences are subsequent objectives. A strong base model need not answer an arbitrary question in chat format. Conversely, a fluent instruction-tuned model does not make the pretraining objective irrelevant: post-training depends on capabilities acquired earlier.

Next-token loss is the immediate optimization target. A fixed held-out loss is an inexpensive, smooth diagnostic when comparing training choices on the same distribution. It is not a complete measure of usefulness. Changing the corpus changes what a given loss means, making downstream evaluation necessary when comparing datasets.

1.2 Resources and choices form a coupled problem

The model lifecycle has at least three cost centers: pretraining, post-training, and inference. Their relative cost varies by project. A heavily used model can consume more inference compute over its lifetime than it consumed in pretraining; this changes the preferred model size. Actual costs depend on the model, hardware, training recipe, and deployment workload.

Before a large run, one must decide architecture, parameter count, initialization, context length, optimizer, learning rate, batch size, schedule, and data mixture. These choices interact. A wider network changes both the optimization dynamics and the efficiency of its matrix multiplications. Longer contexts change memory and attention costs. A better corpus can alter the optimal amount of data. An *ablation* isolates a particular decision sufficiently to establish what caused a performance change.

Developing a pretraining recipe requires a budget for debugging, failed runs, and ablations in addition to the final run. The Smol Training Playbook [3] documents these costs for SmolLM3, a 3B model trained on 11T tokens. Figure 1.1 illustrates how substantial this overhead can be.

Phase	GPUs	Days	GPU Hours
Main pretraining run	384	30	276,480
Ablations (pretraining)	192	15	69,120
Ablations (mid-training)	192	10	46,080
Training reset / debugging	384 / 192	3 / 4	46,080
Total cost	-	-	**437,760**

Figure 1.1: The cost table reproduced in P1:10: 276,480 GPU-hours for the main run, versus 437,760 GPU-hours including ablations and resets/debugging. The development overhead is about 58% of the main-run cost. This is a particular project's accounting, not a universal multiplier.

1.3 Anatomy and approximate parameter count

Let d denote model width, L the number of layers, H the number of attention heads, $a = d/H$ the per-head dimension, F the MLP hidden width, and V vocabulary size. In a standard multi-head attention block, the query, key, value, and output projections contribute approximately $4d^2$ weights. A two-matrix MLP with $F = 4d$ contributes $2dF = 8d^2$. Thus

$$N_{\text{blocks}} \simeq L(4d^2 + 8d^2) = 12Ld^2. \tag{1.3}$$

For SwiGLU, three matrices replace two:

$$\text{MLP}(x) = W_{\text{down}}\left[\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x\right]. \tag{1.4}$$

Choosing $F \simeq 8d/3$ keeps $3dF \simeq 8d^2$, preserving the leading parameter estimate. Implementation-friendly rounding changes the exact count. Normalization gains contribute only $O(Ld)$ terms. Input embeddings add Vd weights; an untied output projection adds another Vd. A lookup embedding has different compute from a dense output projection, so "parameters used for FLOP counting" and "all stored parameters" must be distinguished.

Grouped-query attention shares key/value heads among several query heads. If H_{kv} is the number of key/value heads, the attention projections contain approximately $2d^2 + 2dH_{\text{kv}}a$ weights. Mixture-of-experts models distinguish total parameters from the subset active for each token. These differences become crucial when estimating training cost from N.

The heuristic $d/L \sim 100$ on the slide is a broad design reference, not a requirement. Two models with the same $12Ld^2$ can have different latency and memory behavior: depth creates sequential dependencies, whereas width produces larger matrix multiplications. The chess model intentionally uses a small fixed depth and does not follow that aspect ratio.

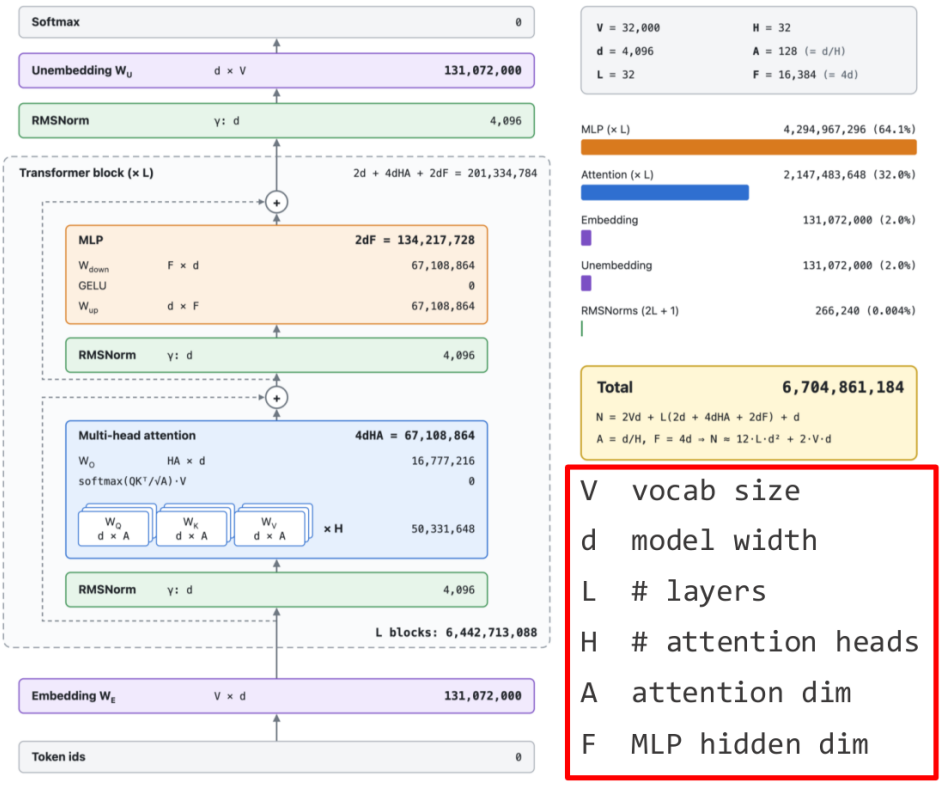


Figure 1.2: The transformer anatomy used on P1:9. Its accounting makes the d^2 terms visible. The approximation $12Ld^2$ assumes ordinary multi-head attention and an appropriately sized MLP; grouped-query attention and mixture-of-experts architectures change the count.

1.4 Why chess is a useful laboratory

Sequences of chess moves provide a low-cost language-modeling problem. Moves are represented by origin and destination squares, such as `e2 e4 e7 e5`, rather than by a large natural-language vocabulary. The vocabulary contains approximately 64 square tokens plus three special tokens (P1:11). Its exact tokenizer specification is not provided here, so the treatment of promotions and special-token identities should be checked in the original code before reproduction.

This setting has abundant sequential data, long-range dependencies, a compact vocabulary, and a meaningful next-token loss. It permits many controlled experiments without a large GPU cluster. The training code can remain chess-agnostic: it receives integer sequences, applies causal attention, and minimizes cross-entropy. The model does not receive an explicit board-state transition engine.

Table 1.1: The base chess configuration (P1:13).

Quantity	Setting
Width d	128
Depth L	4
Head dimension a	32, hence 4 heads
Normalization	RMSNorm
MLP	SwiGLU, hidden width approximately $8d/3$
Position representation	Rotary position embeddings (RoPE)
Stored sequence length	256 tokens
Dominant parameter count	$12 \times 4 \times 128^2 = 786{,}432$



Figure 1.3: The chess illustration from P1:11. The model is trained on the move sequence, not on this rendered board image; the board makes the sequential task interpretable to the reader.

The base configuration processes about 100,000 games per minute on the experimental hardware (P1:13), making sweeps affordable. This runtime depends on the hardware and implementation. A 50,000-game held-out set is used in the learning-rate exercise. The training data are described as shuffled games, making initialization and data-order seeds separate experimental variables.

Splits

Split	Games	Content
train	10,239,287	games of all ratings, shuffled, in 10 files of 1M games and a last file of 239,287 games
validation	50,000	games of all ratings, played after the training games
strong	298,826	games where both players are rated 2200 or more, played after the validation games

Figure 1.4: P1:12: the dataset split. The source pool has 10,239,287 training games, a later 50,000-game validation set, and a separate 298,826-game strong-player split. Temporal separation and rating-based evaluation answer different generalization questions.

1.5 Three token counts that must not be confused

Games contain approximately 135 real tokens on average and are padded to length 256. For G games,

$$D_{\text{slots}} = 256G, \qquad D_{\text{real}} \simeq 135G, \qquad D_{\text{targets}} = \sum_{i,t} m_{i,t}. \tag{1.5}$$

The last quantity can differ slightly from D_{real} because of sequence shifting and special tokens. For 100,000 games, these are about 25.6 million stored positions and 13.5 million real tokens. Both “25M tokens” and “13M tokens” can therefore describe the same experiment under different counting conventions.

Ignoring PAD in the loss prevents learning to predict padding. It does *not* automatically eliminate matrix multiplication at padded positions. Dense padded execution can therefore

process almost twice as many positions as contribute to the loss. A causal mask prevents looking ahead; a loss mask controls which targets count; efficient sequence packing controls wasted computation. These are three different mechanisms.

The chess scaling tables use approximately 135 tokens per game. Their $C = 6ND$ labels should be read consistently with that convention. Unless the implementation avoids padded computation, these labels are useful nominal budgets rather than exact executed hardware FLOPs. This distinction does not invalidate comparisons within a fixed-padding experiment, but it matters for MFU and cross-dataset comparisons.

What to retain. The chess experiments are a laboratory for training methodology. Their numerical learning rates, token ratios, and losses are specific to the tokenizer, architecture, optimizer, data, and accounting convention. The transferable object is the experimental procedure.

Chapter 2

Accounting for compute, memory, and throughput

Lecture slides: P1:16–28.

2.1 Deriving the six-N-D rule

Consider $Y = WX$ with $W \in \mathbb{R}^{I \times J}$ and $X \in \mathbb{R}^{J \times D}$, where D counts token positions in this multiplication. There are $N = IJ$ weights. Counting a multiplication and an addition as two FLOPs gives

$$C_{\text{forward}} \simeq 2IJD, \tag{2.1}$$

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Y} X^\top, \qquad C_{\text{weight gradient}} \simeq 2IJD, \tag{2.2}$$

$$\frac{\partial \mathcal{L}}{\partial X} = W^\top \frac{\partial \mathcal{L}}{\partial Y}, \qquad C_{\text{input gradient}} \simeq 2IJD. \tag{2.3}$$

Summing gives the familiar approximation

$$\boxed{C_{\text{train}} \simeq 6ND.} \tag{2.4}$$

The same parameter is reused across many token positions, which is why both model size and token count enter multiplicatively. Optimizer arithmetic is usually smaller than the matrix-multiplication cost when each update processes many tokens.

This estimate suppresses attention's token-token products, normalization, softmax, activation functions, communication, padding, and recomputation. For ordinary attention, the two main token-token products cost about $4bLs^2d$ FLOPs in a forward pass for batch b and sequence length s. Their forward-and-backward cost is approximately $12bLs^2d$. Dividing by bs gives about $12Lsd$ extra training FLOPs per token. Compared with the $72Ld^2$ block-projection estimate, the ratio is roughly $s/(6d)$. Long contexts can therefore invalidate an unqualified use of $6ND$.

For an MoE model, active weights are a better starting point for arithmetic cost, while total weights govern storage. Routing, shared layers, attention, load imbalance, and communication still require separate treatment. N is not a universally interchangeable number across architectures.

2.2 From FLOPs to wall-clock time

Let P denote the peak FLOP/s of one accelerator in the relevant numerical format, n the accelerator count, and u model FLOPs utilization (MFU). Then

$$t \simeq \frac{C_{\text{model}}}{nPu}, \qquad u = \frac{C_{\text{model}}}{t\,nP}. \tag{2.5}$$

FLOPs is an operation count; FLOP/s is a rate. Keep this distinction explicit when checking units.

The example on P1:23 is $N = 3 \times 10^9$, $D = 11 \times 10^{12}$ and $P = 989 \times 10^{12}$ FLOP/s. Equation (2.4) gives 1.98×10^{23} FLOPs, about 55,612 ideal GPU-hours. At 50% MFU it becomes about 111,224 GPU-hours, or 12.1 days on 384 GPUs. The larger cost in the project table (Figure 1.1) need not imply an error: real runs include different context lengths, attention work, inefficiencies, and interruptions. MFU must be computed using a consistent numerator, precision, and hardware specification.

MFU counts useful model operations relative to peak. Hardware FLOPs utilization (HFU) can count additional executed arithmetic such as recomputation. A job can improve hardware occupancy while doing redundant work; the metrics answer different questions. End-to-end useful throughput also includes downtime and data-loading stalls.

2.3 Arithmetic intensity and the roofline picture

An accelerator has both a compute rate P and a memory bandwidth W. If an operation performs A FLOPs per byte transferred, a simplified roofline bound is

$$P_{\text{achieved}} \leq \min(P, WA). \tag{2.6}$$

For a peak throughput of 989 TFLOP/s and a memory bandwidth of 3.35 TB/s, the crossover is near 295 FLOPs/byte. Large batched matrix multiplications reuse weights over many tokens and can exceed that intensity. Small operations, reductions, transfers, and communication may remain bandwidth- or latency-limited. Thus "pretraining is compute-bound" is a useful tendency for its large matrix multiplications, not a guarantee about every kernel or the entire job.

2.4 Where the memory goes

In a straightforward FP32 implementation of Adam, every parameter needs one value for the weight, one for its gradient, and two for the first and second moment estimates:

$$M_{\text{persistent}} \simeq 4N + 4N + 4N + 4N = 16N \text{ bytes}. \tag{2.7}$$

For the approximately 0.8M-parameter chess model this is about 12.8 MB, consistent with the 13 MB baseline in P1:20. This estimate excludes activations, buffers, allocator reservations, framework overhead, and temporary optimizer allocations. Mixed precision changes the breakdown and may retain FP32 master weights and optimizer states.

Activations are intermediate values needed to differentiate the model. Their storage grows with microbatch size, sequence length, width, and depth. Naive attention additionally stores arrays of order $bHLs^2$. Memory-efficient attention and activation checkpointing modify these scalings. The PyTorch memory-visualization reference [29] provides a way to inspect allocation lifetimes instead of inferring them from one aggregate number.

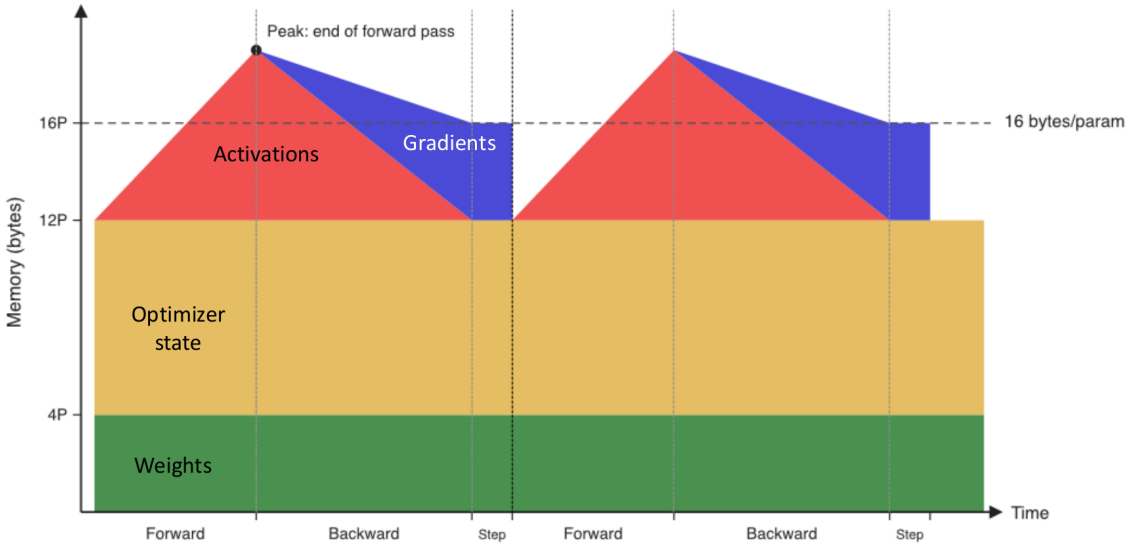


Figure 2.1: The memory lifecycle on P1:20, assuming Adam and FP32. Activations accumulate during the forward pass; backward frees many saved activations while materializing gradients. Persistent weights and optimizer moments remain. The actual peak depends on overlap, tensor lifetimes, and implementation.

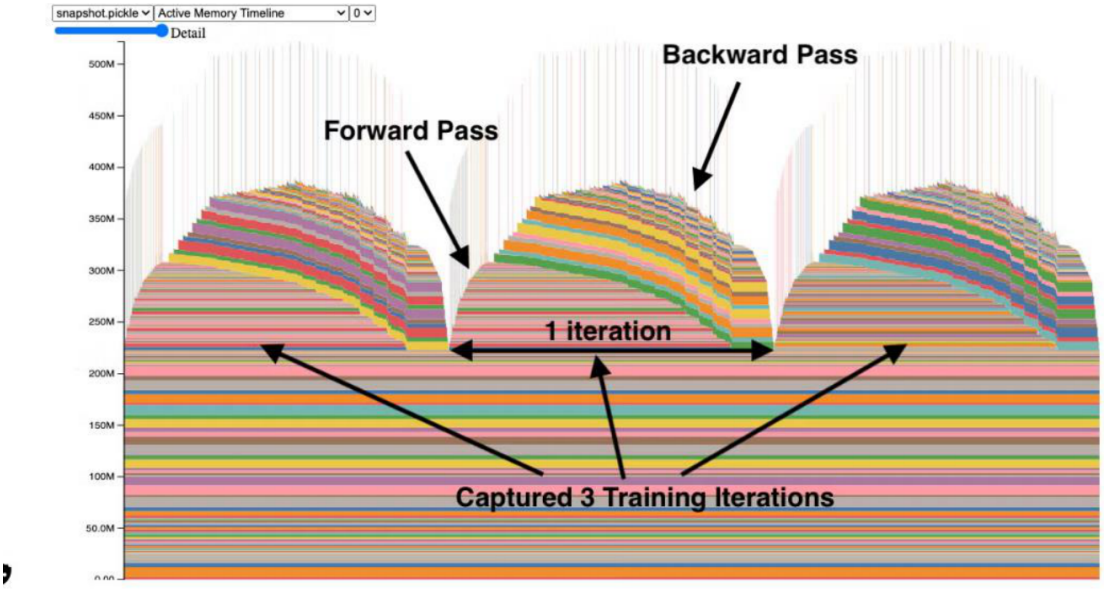


Figure 2.2: P1:21: an allocation timeline from the memory-profiling discussion. Such a trace reveals which tensors coexist at the peak and which allocations persist between steps. A single total-memory reading cannot reveal these lifetimes.

2.5 Exercise 1: measuring the device, not guessing

Measure memory and throughput over batch sizes from 1 to roughly 1,000 games. Several forward-and-backward passes are required: the first pass can initialize kernels, allocate optimizer states, or trigger compilation. On an asynchronous device, timings require synchronization before and after the measured region. Measure both a large matrix multiplication and the actual training step, because the former is an empirical upper bound rather than a prediction of application throughput.

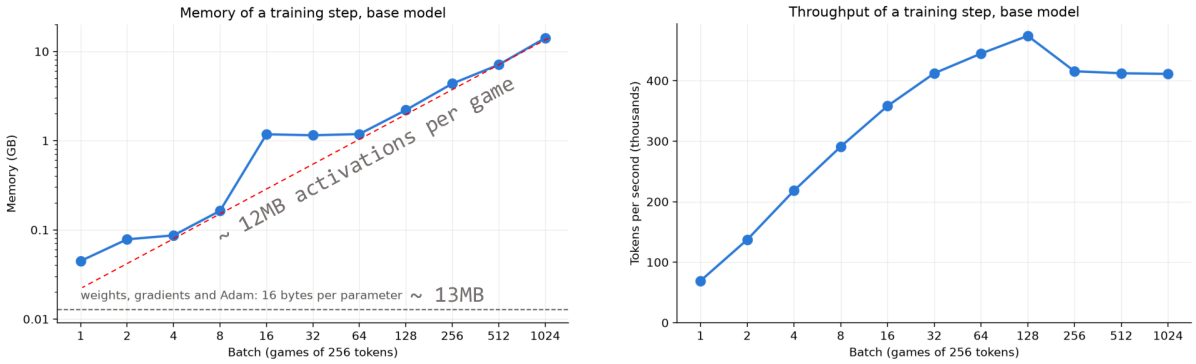


Figure 2.3: P1:26: memory consumption (left) and token throughput (right) versus microbatch size. The reported activation cost is about 12 MB per game, while weights, gradients, and Adam states account for only about 13 MB. Throughput initially rises, then saturates and fluctuates. The vertical throughput axis is thousands of tokens per second; this unit must be distinguished from games per second.

The memory curve measures driver-allocated memory on Apple Silicon, which need not vary linearly with batch size. Allocation granularity and cached buffers create plateaus and jumps. Allocated, reserved, and peak live tensor memory are related but distinct quantities. The commands on P1:25 also refer to different accounting interfaces: CUDA's peak allocated memory and the MPS driver's allocated memory should not be assumed to measure exactly the same thing.

The throughput curve identifies a useful *hardware* range. Below saturation, a larger microbatch can use idle parallelism. Beyond saturation, additional memory may buy no speed. This says nothing yet about the *statistically* best optimizer batch size; that question requires validation-loss experiments.

2.6 Microbatch, accumulation, and global batch

Let b be the number of sequences in one microbatch, K the accumulation count, and n_{dp} the number of data-parallel replicas. The global optimizer batch is

$$B_{\text{seq}} = bKn_{\text{dp}}, \qquad B_{\text{slots}} = sbKn_{\text{dp}}. \tag{2.8}$$

The model weights stay fixed while gradients from K microbatches are accumulated; then the optimizer steps once. Averaging gradients reproduces the corresponding full-batch gradient in exact arithmetic if weighting is correct and no batch-dependent operation changes the calculation. With variable valid-token counts, average by total valid tokens, not by an unweighted average of microbatch means. Floating-point order and stochastic layers can introduce small differences.

For a desired batch of 16,000 games when only 1,000 fit at once, use 16 microbatches. Gradient clipping should normally occur after accumulation and any precision unscaling. The learning-rate schedule advances per optimizer step unless deliberately defined in tokens; changing accumulation without updating schedule bookkeeping changes the experiment.

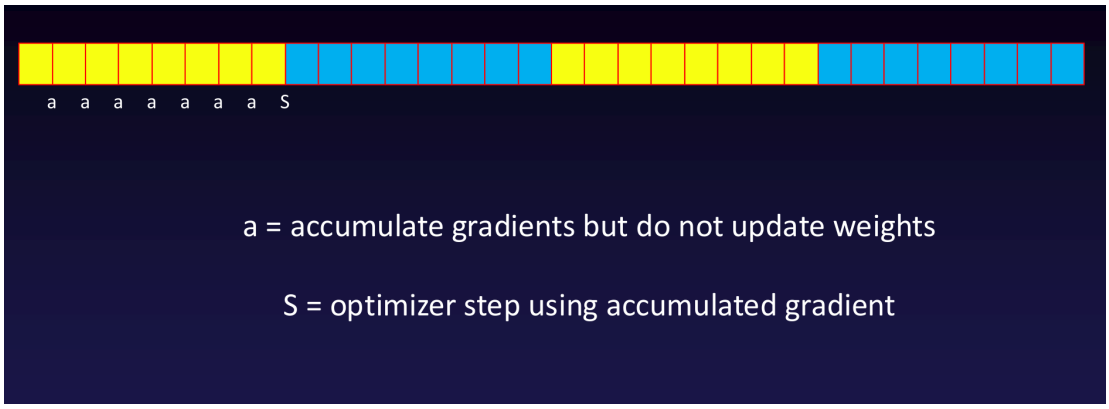


Figure 2.4: The accumulation timeline from P1:27. The small marks denote gradient accumulation; S denotes an optimizer step. Increasing accumulation changes the optimizer batch without requiring all activations to coexist in memory.

2.7 Width changes utilization

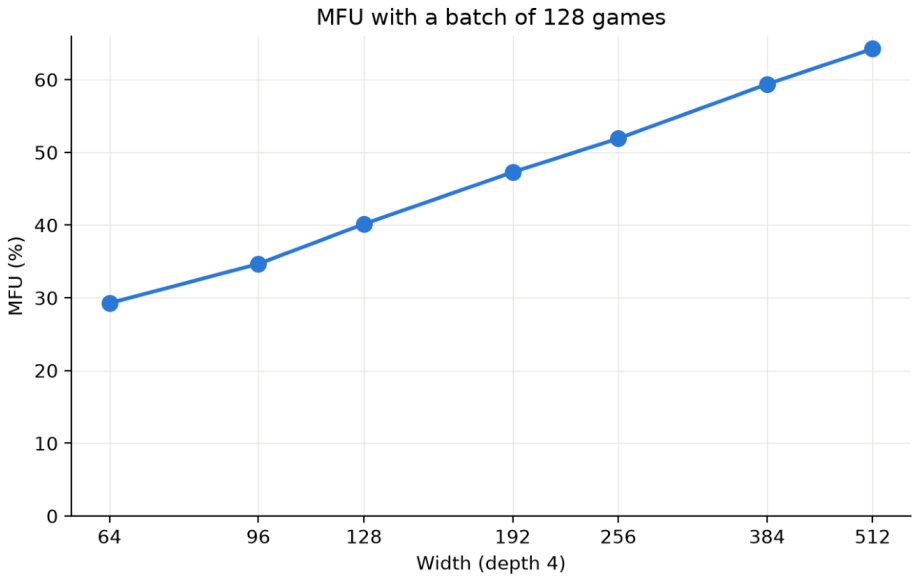


Figure 2.5: P1:28: measured MFU rises from roughly 29% at width 64 to 64% at width 512, at depth 4 and a batch of 128 games. The peak FLOP/s denominator is estimated on the Apple Silicon device used for this experiment. Larger models still require more time; they merely use the device more efficiently.

Wider models generate larger matrix multiplications, which can reduce the fraction of time lost to launch overhead and improve hardware utilization. Consequently, optimality under a FLOP budget need not equal optimality under a wall-clock budget. A more expensive mathematical model can sometimes make better use of the hardware. All later scaling results should be interpreted with their chosen budget in mind.

2.8 Reading power laws

If $Y = cX^{\alpha}$, then $\log Y = \log c + \alpha \log X$. The slope of a log-log graph is α, regardless of logarithm base. A straight segment is evidence for a power-law approximation over that range, not proof of an asymptotic law. If $Y = E + cX^{-\alpha}$, it is $\log(Y - E)$, not generally $\log Y$, that is

exactly linear.

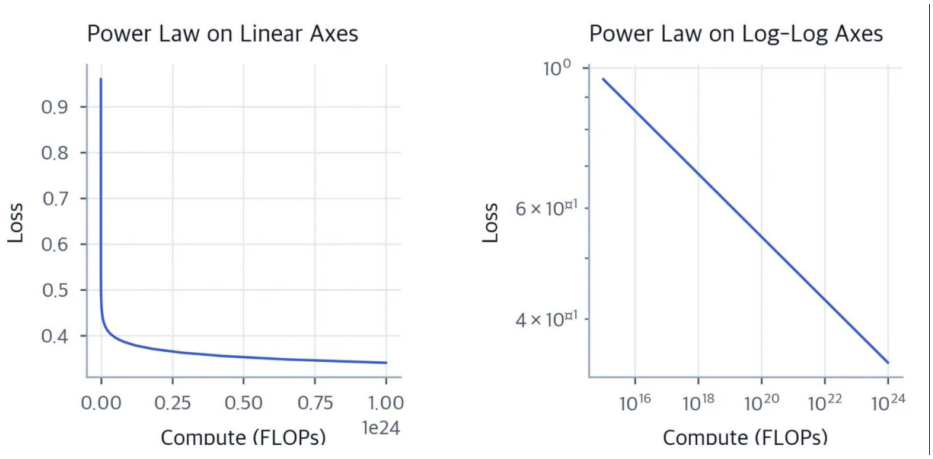


Figure 2.6: The linear-axis and log-log illustrations on P1:24. Axis transformations expose multiplicative scaling, but offsets, limited ranges, and uncertain measurements still matter.

What to retain. Measure the actual training step. Keep stored parameters, active parameters, useful tokens, processed positions, model FLOPs, hardware FLOPs, and wall-clock time distinct. An efficiency metric is meaningful only with its accounting convention.

Chapter 3

Optimization: AdamW and its timescales

Lecture slides: P1:18, 30–36.

3.1 From stochastic gradient descent to Adam

For a minibatch loss $\mathcal{L}_t$, write $g_t = \nabla_\theta \mathcal{L}_t(\theta_t)$. SGD updates $\theta_{t+1} = \theta_t - \eta_t g_t$. Adam [14] instead maintains elementwise exponential moving averages:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{3.1}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \tag{3.2}$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{3.3}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}. \tag{3.4}$$

All divisions and square roots are coordinatewise. The first moment smooths directions; the second estimates the scale of recent squared gradients. v_t is a *second raw moment*, not a variance: a variance would subtract the squared mean. Bias correction compensates for initializing the averages at zero, under the corresponding stationarity interpretation. It does not eliminate noise or make the estimates instantly reliable.

To understand the learning rate, suppose a coordinate has constant nonzero gradient g and $\epsilon \ll |g|$. Then $\widehat{m} \approx g$, $\widehat{v} \approx g^2$, and

$$\frac{\widehat{m}}{\sqrt{\widehat{v}} + \epsilon} \approx \operatorname{sign}(g), \qquad |\Delta\theta| \approx \eta. \tag{3.5}$$

Under these assumptions, Adam's learning rate approximates the magnitude of a per-coordinate displacement. The precise identity underlying this interpretation is $\sqrt{g^2} = |g|$, not g. With noisy, changing gradients the normalized update need not equal one. A coordinate with rapidly alternating signs can have small m and appreciable v, hence a much smaller net update.

3.2 Why weight decay must be decoupled

Adding $\lambda \|\theta\|^2/2$ to the objective produces gradient $g + \lambda\theta$. For ordinary SGD,

$$\theta_{t+1} = \theta_t - \eta_t(g_t + \lambda\theta_t) = (1 - \eta_t\lambda)\theta_t - \eta_t g_t. \tag{3.6}$$

Thus an L_2 penalty is equivalent to multiplicative decay in this setting. For Adam, putting $g + \lambda\theta$ inside the moment estimates changes both m and v. The penalty is then transformed by a coordinate-dependent, history-dependent preconditioner. It no longer creates uniform shrinkage of all weights.

AdamW [16] applies the adaptive update to the task gradient and applies shrinkage separately:

$$\boxed{\theta_{t+1} = (1 - \eta_t\lambda)\theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}.} \tag{3.7}$$

"Decoupled" means decoupled from adaptive gradient preconditioning. In the common convention used here, decay is still multiplied by the learning rate. Different software conventions can use a coefficient with different semantics; copy an update equation, not just a number named "weight decay."

3.3 Interpreting the beta parameters

Unrolling a moving average gives weights proportional to $(1 - \beta)\beta^k$ for a contribution k steps old. Its exact e-folding time is

$$\tau_\beta = -\frac{1}{\log \beta} \simeq \frac{1}{1 - \beta} \quad (\beta \text{ close to } 1). \tag{3.8}$$

After one such timescale, an old contribution has decayed by a factor near e^{-1}; it has not vanished completely. An averaging time of ten steps therefore does not mean that the estimate is fully renewed after ten steps.

Table 3.1: Approximate averaging times in optimizer steps.

β	$1/(1 - \beta)$	Interpretation
0.9	10	Relatively rapid first-moment response
0.95	20	Shorter second-moment memory
0.975	40	Both moments in the ML4 example
0.999	1,000	Much longer second-moment memory

A practical starting point for pretraining is $(\beta_1, \beta_2) = (0.9, 0.95)$, which uses a shorter second-moment memory than $(0.9, 0.999)$. The shorter second-moment memory tracks changing gradient scales more quickly. This is a practical recipe, not a theorem and not a statement that every pretraining run needs the same betas. Mistral Large 4, for example, uses $(0.975, 0.975)$ [23].

The averaging time in *tokens* is about $B_{\text{tok}}/(1 - \beta)$, so a change in batch size changes the temporal meaning of a fixed beta. To preserve exponential forgetting per token when the batch grows by a factor k, a natural relation is $\beta' \approx \beta^k$. This observation explains why batch-size transfer can require optimizer retuning as well as a learning-rate adjustment; it is not a blanket prescription for all settings.

Paper connection: Adam spikes. Bai et al. [1] examine an instability that can arise from Adam's adaptive preconditioner itself. When instantaneous squared gradients become small relative to the stored second moment, v can decay while lagging the current dynamics. The effective preconditioned curvature can cross a stability threshold. Their quadratic analysis and network experiments show why constant curvature alone does not exclude an Adam spike. This supports monitoring optimizer state and gradient dynamics, rather than blaming every spike on a bad example. It does not prove that the same mechanism caused each spike in the chess figures.

3.4 Weight decay when training for roughly one pass

Weight decay can help even when a web-scale model makes only about one pass through a huge corpus. The simple classical story, "it prevents fitting the same examples too closely," is incomplete. Training once does not logically rule out memorization, distributional overfitting, or repeated content within a corpus. It does, however, motivate looking at optimization dynamics in addition to traditional regularization.

Let $q_t = \widehat{m}_t/(\sqrt{\widehat{v}_t} + \epsilon)$ and, for $\lambda > 0$, define

$$u_t = -q_t/\lambda. \tag{3.9}$$

Equation (3.7) becomes

$$\theta_{t+1} = (1 - \eta_t\lambda)\theta_t + \eta_t\lambda u_t. \tag{3.10}$$

For constant η and $0 < \eta\lambda < 1$, this is an EMA toward a moving target u_t. The minus sign in the definition of u_t is necessary; a positive definition followed by a plus update would reverse the gradient direction. The decay-memory timescale is

$$\tau_{\text{wd}} = -\frac{1}{\log(1 - \eta\lambda)} \simeq \frac{1}{\eta\lambda}. \tag{3.11}$$

For varying learning rates, the old-weight contribution is multiplied by $\prod_t(1-\eta_t\lambda)$, approximately $\exp(-\lambda\sum_t \eta_t)$. Consequently, schedule shape and training length also affect effective forgetting.

Two complementary papers. D'Angelo et al. [7] argue that the benefit of weight decay in modern training can come from its modification of optimization dynamics. Their account distinguishes repeated-pass vision training from nearly single-pass language modeling, where stability and stochastic bias–variance tradeoffs become central. Wang and Aitchison [33] formalize the EMA view and study how its preferred timescale transfers across dataset and model sizes. Their empirical finding concerns a roughly stable timescale measured in epochs, not a universal optimal value of λ. Both papers help explain why a useful decay coefficient can remain nonzero without a classical overfitting curve.

Further explanation. If D is a token horizon and B_{tok} a batch, an epoch-like timescale is $\tau_{\text{epochs}} \simeq B_{\text{tok}}/(\eta\lambda D)$. Keeping this fixed while changing D, B, or η imposes a joint relation on the hyperparameters. At fixed D and batch, reducing matrix learning rates by a factor r requires increasing decay by r to preserve $\eta\lambda$. This will reappear in the width-transfer discussion.

The chess code uses decay 0.1 on matrix weights and excludes one-dimensional normalization gains. This parameter grouping is a convention motivated by their different role, not a mathematical equivalence between "one-dimensional" and "should never be decayed." Embeddings and tied weights deserve deliberate treatment in a production implementation.

3.5 The training loop and faithful restarts

Each optimizer step sets the current learning rate, obtains input/target pairs shifted by one token, computes logits with causal attention, evaluates masked cross-entropy, backpropagates, clips gradients if configured, and updates the model. Gradients must be cleared at the correct accumulation boundary. The code shown uses BF16 autocasting for supported devices and computes the loss from float logits.

Global norm clipping transforms g into

$$\widetilde{g} = g \min\left(1, \frac{c}{\|g\|_2}\right). \tag{3.12}$$

The chess experiments use a clipping threshold near $c = 1$. Log the *pre-clipping* norm, otherwise large events are hidden. Clipping a gradient does not directly bound the Adam parameter-update norm because moment history and coordinatewise normalization intervene.

A faithful checkpoint contains weights *and* optimizer state (m, v, and step count). For reproducible continuation, also preserve schedule progress, data position or sampler state, random-generator states, precision-scaler state if applicable, and configuration. Loading weights alone is a new optimization trajectory, not a faithful resume.

What to retain. AdamW introduces interacting timescales: moment averaging, decay, warmup, and the total run length. Starting values are useful, but understanding those timescales is what makes transfer and debugging possible.

Chapter 4

Learning-rate schedules and reusable runs

Lecture slides: P1:36–43.

4.1 Why warm up and why cool down?

Warmup limits early updates while optimizer statistics and internal activation scales settle. Adam's bias correction removes a particular initialization bias; it does not remove uncertainty in early statistics or guarantee stability of the network. Starting with a smaller learning rate is usually cheap relative to a long run.

Cooldown reduces the size of stochastic updates near the end. A useful local model is a quadratic loss with noisy gradients:

$$\mathcal{L}(w) = \tfrac{1}{2}h(w - w_*)^2, \quad w_{t+1} - w_* = (1 - \eta h)(w_t - w_*) - \eta\xi_t, \tag{4.1}$$

where $\mathbb{E}\xi_t = 0$ and $\mathbb{E}\xi_t^2 = \sigma^2$. For $0 < \eta h < 2$, the stationary variance is

$$\operatorname{Var}(w - w_*) = \frac{\eta^2\sigma^2}{1 - (1 - \eta h)^2} \simeq \frac{\eta\sigma^2}{2h}. \tag{4.2}$$

Smaller steps can reduce a noise floor. This calculation is an explanatory SGD toy model, not a proof of WSD's behavior in a transformer. It also clarifies the role of sharpness: high curvature limits stable step size. A sharp region does not, by itself, justify a larger step, even if its gradient is large.

4.2 Cosine with warmup

Let S be the intended number of optimizer steps, W the warmup length, $\eta_{\max}$ the peak rate, and r the final-to-peak ratio. A continuous convention is

$$\eta(t) = \begin{cases} \eta_{\max} t/W, & 0 \le t < W, \\ \eta_{\max}\left[r + \dfrac{1-r}{2}\left(1 + \cos\left(\pi\dfrac{t-W}{S-W}\right)\right)\right], & W \le t \le S. \end{cases} \tag{4.3}$$

The implementation in P1:37 decays to 10% of the peak learning rate. Discrete code must decide whether the first optimizer step is indexed by 0 or 1, and should check the actual first and last learning rates. A schedule defined in games or tokens must be converted consistently when batch size changes.

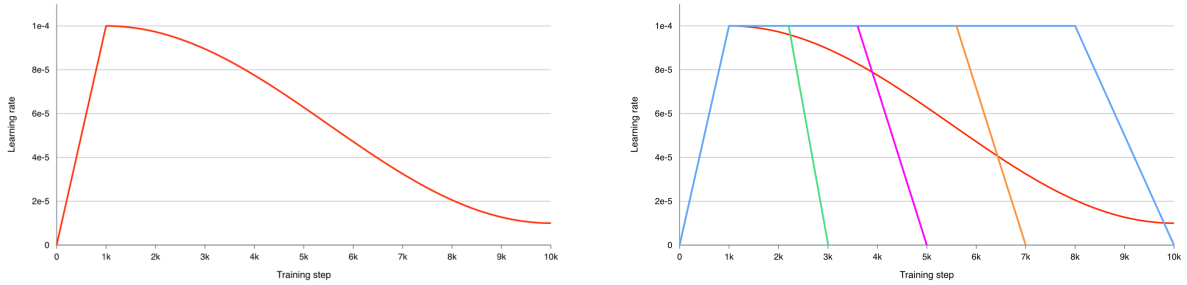


Figure 4.1: P1:37–38: cosine with warmup (left), and warmup–stable–decay schedules with several endpoints (right). WSD shares a long plateau across run lengths; cosine changes its trajectory when its planned endpoint changes.

4.3 Warmup–stable–decay (WSD)

With a cooldown occupying a fraction q of the total horizon, a simple WSD schedule is

$$\eta(t) = \begin{cases} \eta_{\max} t/W, & 0 \le t < W, \\ \eta_{\max}, & W \le t < (1-q)S, \\ \eta_{\max}\left[1 - (1-r)\dfrac{t-(1-q)S}{qS}\right], & (1-q)S \le t \le S. \end{cases} \tag{4.4}$$

Assume $W < (1-q)S$. This schedule uses linear decay, although other decay shapes are possible. In Exercise 2, the warmup is 10,000 games and the cooldown is 20% of the chosen horizon.

The main advantage is flexibility. If the training horizon changes, a plateau checkpoint remains useful as a branching point. A cosine run stopped halfway has not followed the cosine schedule that would have been chosen for that shorter horizon. WSD does not magically finish with no extra compute: a proper branch still needs its cooldown budget. If an endpoint is G games and cooldown is 20%, branch at $0.8G$ and use another $0.2G$ games.

4.4 Exercise 2: what the curves actually establish

Compare a width-128 base model at 500,000 and one million games. The long cosine run and long WSD run both finish at validation loss 1.283 in the displayed experiment. At 500,000 games, the planned cosine run finishes at 1.359 and the WSD branch at 1.355. These printed results support comparable performance in this setup. The 0.004 difference is not evidence of a general ranking without repetitions and uncertainty estimates.

The key comparison is between *completed schedules at the same endpoint*. Comparing an uncooled plateau model with a cooled model confounds optimization progress with the schedule phase. Likewise, one should not select the peak learning rate from the lowest plateau loss when the intended metric is loss after cooldown.

4.5 Branching turns one long trajectory into several experiments

P1:42 shows a 3.2-million-game run with several checkpoint branches. Each branch is cooled down to a specified endpoint. The displayed final validation losses are:

100k	200k	400k	800k	1.6M	3.2M
1.761	1.553	1.406	1.317	1.259	1.203

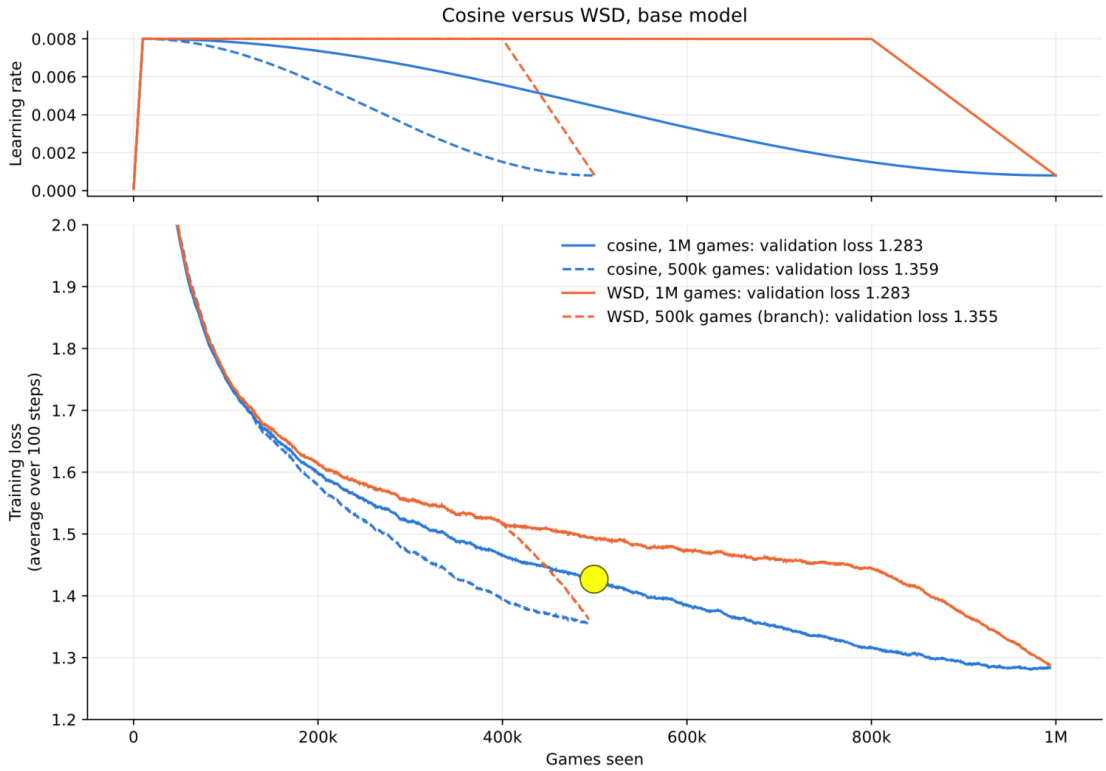


Figure 4.2: P1:41: schedule values above, training-loss traces below, final validation losses in the legend. The orange plateau appears worse than the blue cosine curve during much of training, but cooldown closes the gap. The highlighted midpoint of the long cosine run is not a properly completed short run.

These values are for the illustrated peak rate 0.004, not the best rate at every horizon. A later experiment retunes the peak rate separately.

To see the compute saving, consider horizons $G, 2G, 4G, \ldots, G_{\max}$ and cooldown fraction q. Independent runs cost nearly $2G_{\max}$ games in total. A shared trunk reaching $(1-q)G_{\max}$ plus all decays costs approximately $(1-q)G_{\max} + 2qG_{\max} = (1+q)G_{\max}$. With $q = 0.2$, this is about 60% of the independent-run cost, ignoring warmup details and finite-series endpoints.

The endpoints are statistically correlated because they share parameters and data history. They are not independent random seeds. This matters when fitting a scaling law or estimating uncertainty from branched trajectories.

4.6 Alternatives and the scope of the result

DeepSeek LLM [8] studies stepped schedules and joint scaling of learning rate and batch size. Cosine decay is a convention that can be tested, not a mathematical requirement. Schedule comparisons must match token budget, warmup definition, final rate, and the amount of tuning given to each family.

What to retain. Cooldown is part of the experiment. WSD's value is both optimization performance and experimental reuse: it allows multiple properly finished endpoints to share a common training trunk.

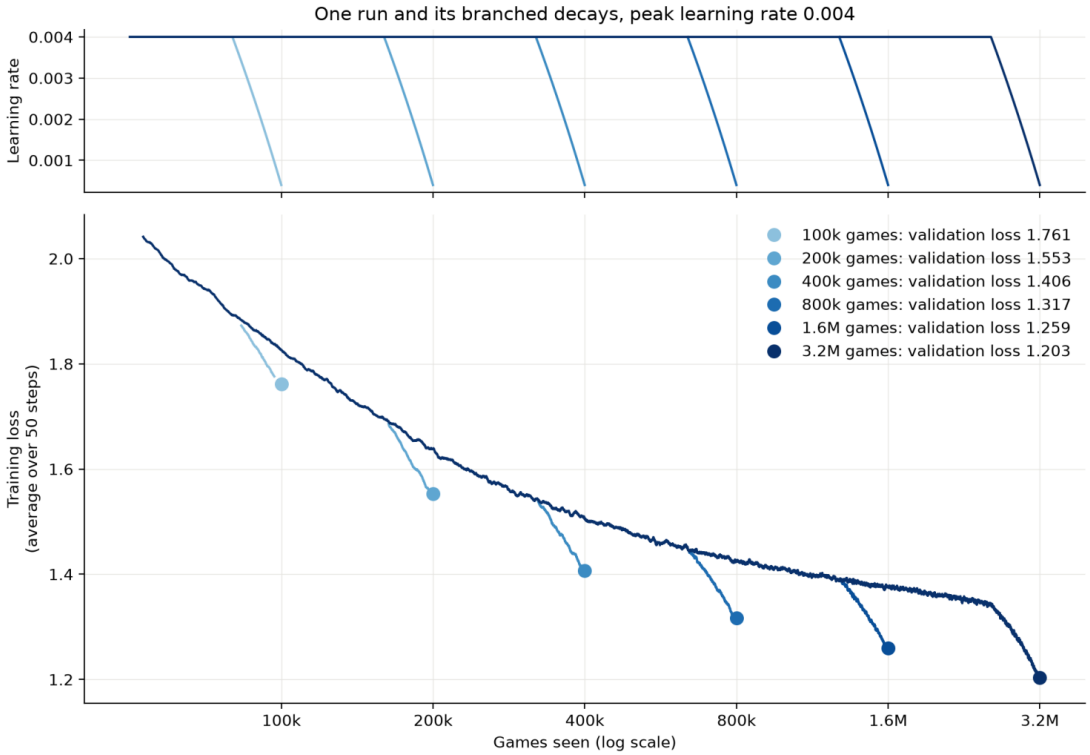
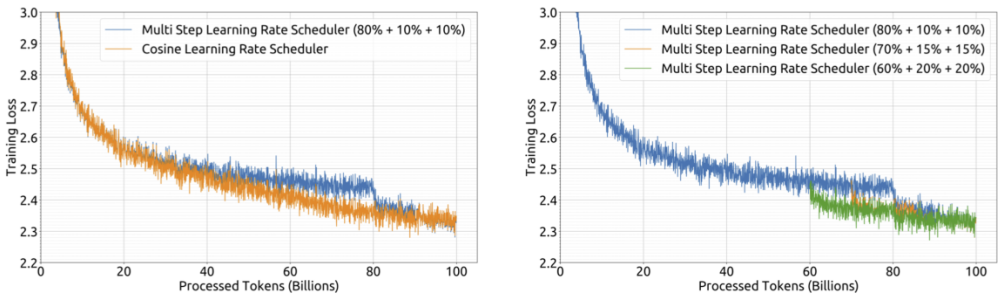


Figure 4.3: P1:42 (repeated on P1:61): a shared WSD trunk and its cooldown branches. The horizontal axis is logarithmic in games seen. The benefit of cooldown grows at the longer endpoints in this example, even when the plateau training loss has slowed.



(a) Multi-step v.s. cosine learning rate decay (b) Different proportions of multi-step stages

Figure 1 | Training loss curves with different learning rate schedulers or different parameters for schedulers. The model size is 1.6 billion parameters, trained on a dataset of 100 billion tokens.

Figure 4.4: The DeepSeek LLM schedule comparison reproduced on P1:43. Stepwise schedules are another way to reserve substantial learning-rate reduction for late training. The model and dataset differ from the chess experiment.

Chapter 5

Learning rate, batch size, and training horizon

Lecture slides: P1:44–66.

A learning rate is not intrinsically good: it is good for a particular model, optimizer, batch size, data distribution, and number of updates. Vary these ingredients one at a time before combining their effects.

5.1 Exercise 3a: find the useful learning-rate range

Use the small four-layer chess transformer, a fixed batch size, a WSD schedule, and 100,000 training games. Evaluate on a separate set of 50,000 games. The training input contains 25.6 million padded positions but approximately 13.5 million non-padding tokens. Keep the warmup, cooldown fraction, initialization, and data order fixed for a controlled first comparison.

Sweep the peak learning rate geometrically, for example

$$\eta_k = 10^{-3} 2^k,$$

then refine the promising interval using factors of $\sqrt{2}$. A logarithmic grid is appropriate because useful learning rates usually span multiplicative ranges. Selecting the best run among many also introduces selection noise, so repeat the promising configurations across seeds.

The low-loss region need not coincide with the fastest early decrease in training loss. Final validation losses in P1:48 are approximately 1.643 at 0.0113, 1.647 at 0.0226, 2.434 at 0.032, and 2.694 at 0.064. The last two are outside the useful range. The correct response is to inspect the whole trajectory, including gradient norms and the cooldown, rather than to rank runs by their first few updates.

5.2 How much should we trust a single run?

There are at least three distinct sources of variation: numerical nondeterminism on the same nominal seed; different initial weights; and different data orders or sampled data. In these measurements, initialization matters much more than the residual variation from rerunning the same seed. Data variation adds another contribution. This is an observation for these experiments, not a universal ordering.

A convenient local model is a quadratic in log learning rate:

$$\mathcal{L}_{\text{val}}(\eta) \simeq a\big(\log \eta - c\big)^2 + \mathcal{L}_{\min}, \qquad a > 0, \tag{5.1}$$

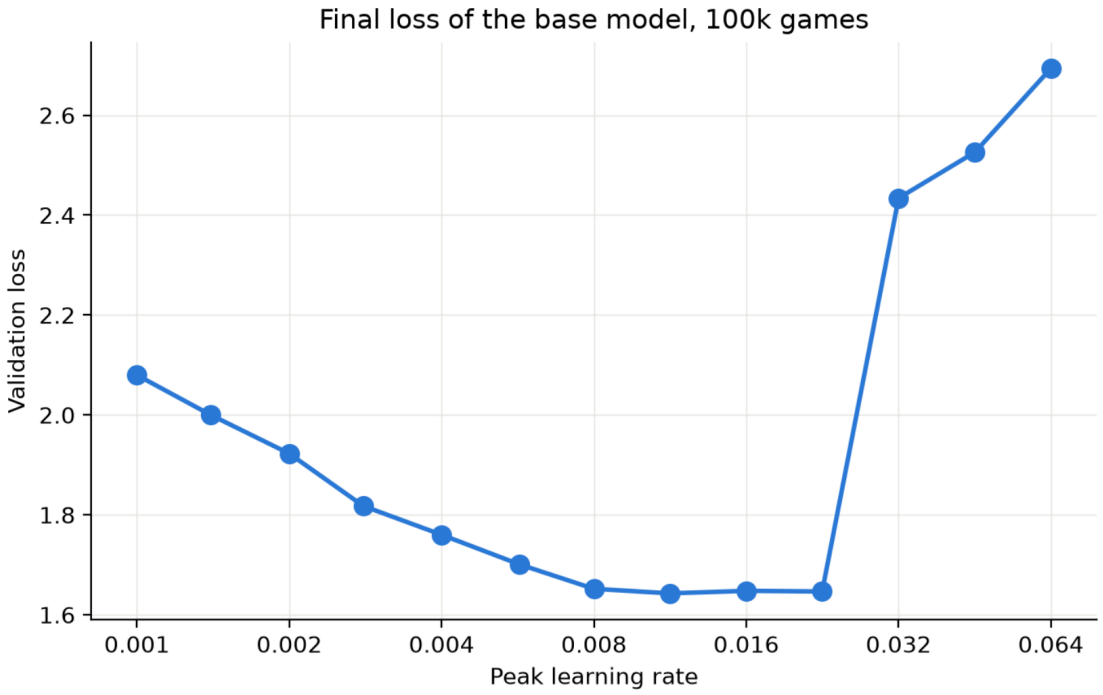


Figure 5.1: Learning-rate sweep from P1:47. The best region is broad, roughly 0.008–0.0226 in this particular experiment, followed by a sharp deterioration near 0.032. These values are specific to the model and training horizon.

so the estimated optimum is $\eta^* = e^c$. Fit the best point and its two neighbors. Three points determine a parabola, but do not establish that the curve is quadratic over a wide interval. The fit is especially fragile on a flat bottom or next to a divergence boundary. Repeating the sweep, fitting more nearby points, and reporting an interval of nearly equivalent learning rates are more informative than many digits in η^*.

5.3 Batch size: hardware efficiency versus statistical efficiency

The batch size controls how many examples contribute to one update. For independent samples with gradients g_i,

$$\bar{g}_B = \frac{1}{B}\sum_{i=1}^{B} g_i, \qquad \mathbb{E}[\bar{g}_B] = g, \qquad \mathrm{Cov}(\bar{g}_B) = \frac{\Sigma}{B}.$$

A larger batch reduces gradient noise. It also performs fewer optimizer updates for the same number of examples. These effects compete. At a fixed data budget, doubling B halves the number of updates; at a fixed step count, it doubles the data budget. The two comparisons answer different questions.

McCandlish et al. [19] propose a gradient-noise scale as a useful predictor of the largest efficient batch. A simplified expression is

$$B_{\mathrm{noise}} = \frac{\mathrm{tr}\,\Sigma}{\|g\|^2}. \tag{5.2}$$

When the stochastic component dominates, averaging more examples can be valuable. Once it is small relative to the mean direction, further averaging yields diminishing returns. The paper develops more careful curvature-dependent versions and tests the idea across several domains, including language modeling. The paper was published in 2018.

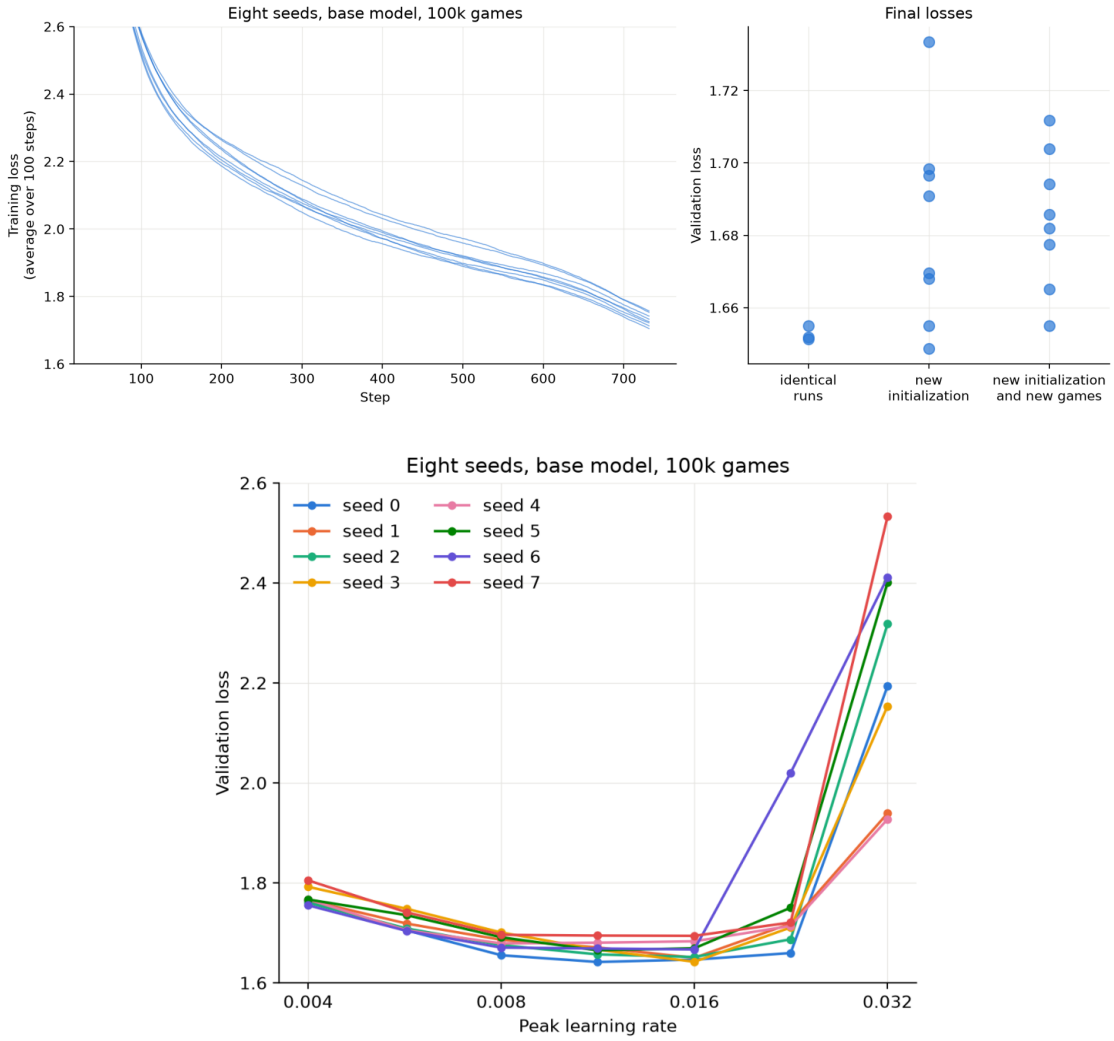


Figure 5.2: Repeated runs, P1:50–51. The upper panels separate sources of variability. The lower plot shows eight seed-dependent learning-rate curves; fitting their nearby minima gives a distribution of optimum estimates.

Further explanation. "Critical batch size" is an empirical crossover, not a hard mathematical ceiling. It changes during training, depends on the target loss, and interacts with optimizer settings. Correlated examples, variable sequence lengths, and token masking also weaken the simple independent-sample argument.

5.4 Exercise 3c: tune batch size and learning rate jointly

Sweep batch sizes from 16 to 1024 and retune the learning rate for each one. This is essential: holding the learning rate fixed could make a batch size look poor merely because the associated optimizer setting is inappropriate. At a fixed training budget, very large batches eventually hurt because too few updates remain.

The rough rule $\eta \propto \sqrt{B}$ is a local empirical guide here. It is not the same as the often-discussed linear scaling rule for SGD, and neither rule should be applied without its assumptions. A larger batch changes the number of updates per epoch and therefore the interpretation of momentum and weight-decay timescales. For example, a moment horizon of $1/(1-\beta_2)$ updates corresponds to approximately $B/(1-\beta_2)$ examples.

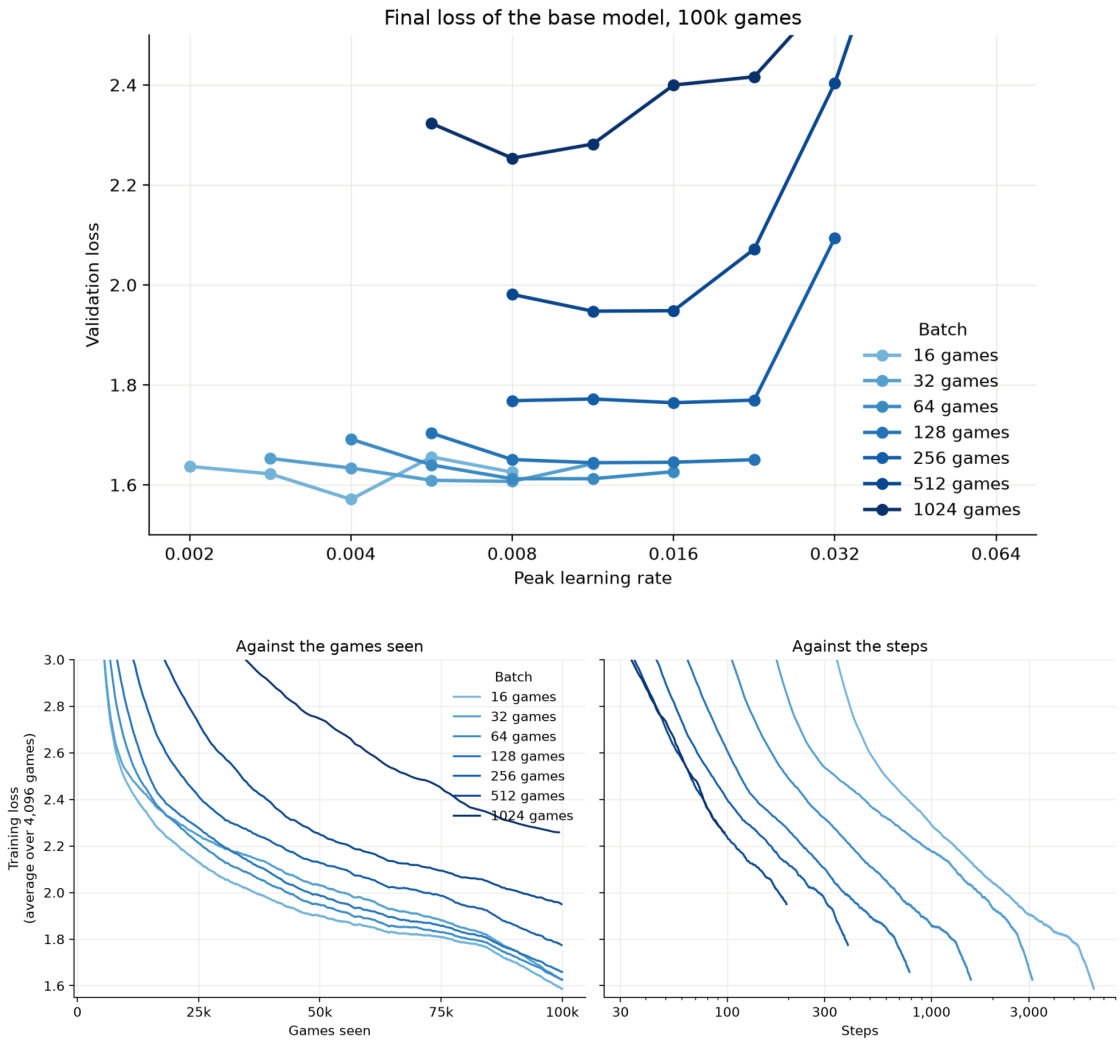


Figure 5.3: Chess batch-size experiment, P1:55–56. Loss must be compared both against examples processed and against optimizer steps. Larger batches can look advantageous on one horizontal axis and inefficient on the other.

The engineering decision is consequently two-dimensional: select a statistically useful *global* batch, then implement it using a microbatch and accumulation pattern that uses the GPU well. Throughput alone cannot determine the best global batch.

5.5 The optimal learning rate depends on the horizon

A longer training horizon allows smaller, less noisy updates to accumulate, so the best peak learning rate tends to decrease. Increasing the number of training tokens eightfold can therefore require a substantially lower peak rate.

Bjorck et al. [4] study this dependence over hundreds of controlled runs and propose an approximate power law. Their reported exponent near -0.32 means that multiplying the horizon by eight roughly halves the optimal learning rate. The exponent depends on the setting; in particular, their small models do not all follow exactly the same exponent. Width transfer with μP does not itself solve horizon transfer.

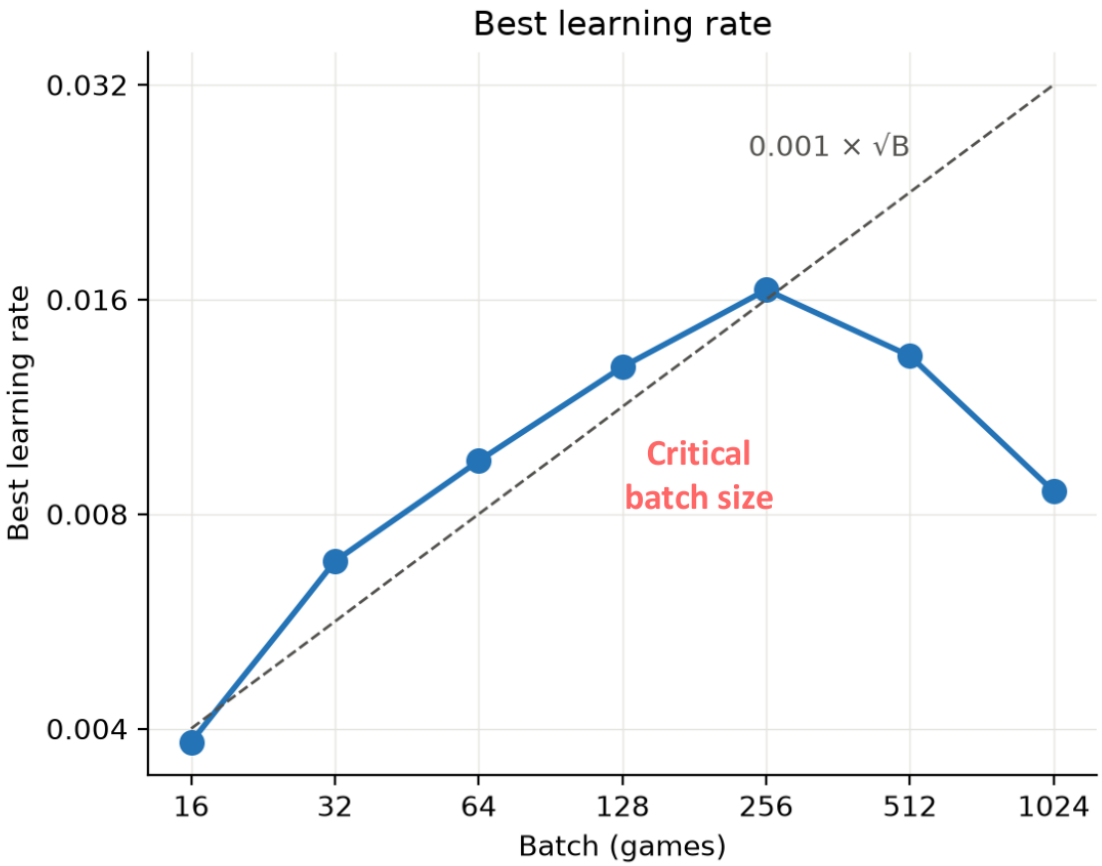


Figure 5.4: P1:57: the optimal learning rate initially grows approximately with the square root of batch size, then turns down. A batch size of 128 provides a practical setting below the inefficient large-batch regime in this experiment.

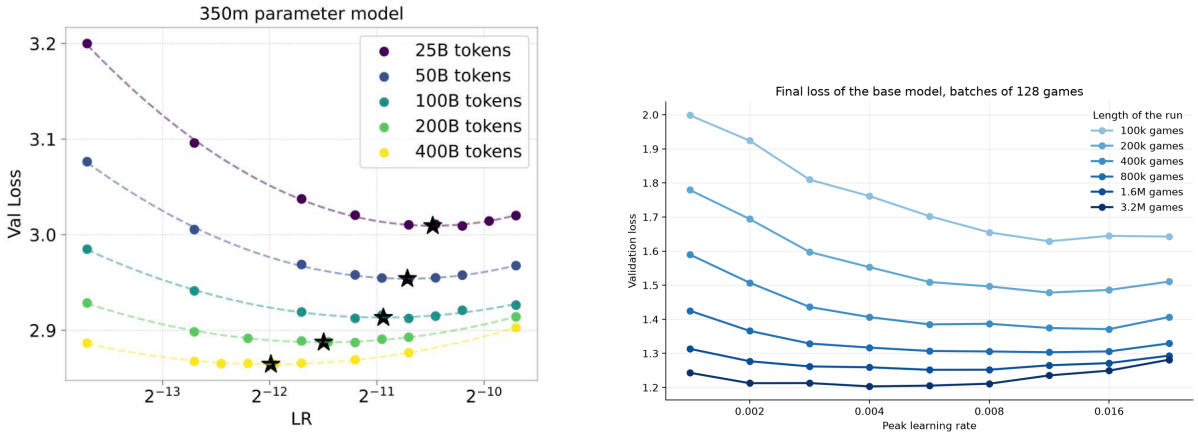


Figure 5.5: Horizon-scaling results from the paper cited on P1:59 and the chess sweeps on P1:62. As the training horizon grows, the favorable learning-rate region moves left.

5.5.1 Exercise 3d: fitting the horizon rule

Keep the batch size at 128. Train horizons ranging from 100,000 to 3.2 million games, and use matched WSD endpoints. For each horizon, fit the local minimum using Equation (5.1). Then regress $\log \eta^*$ against $\log G$, where G is the number of training games. The fitted relation is approximately

$$\boxed{\eta^*(G) \simeq 0.0156 \left(\frac{G}{100{,}000} \right)^{-0.36}}. \tag{5.3}$$

The displayed uncertainty on the fitted exponent is about 0.02. Doubling the horizon multiplies the recommended learning rate by $2^{-0.36} \simeq 0.779$: a reduction of about 22%. An eightfold increase gives $8^{-0.36} \simeq 0.473$.

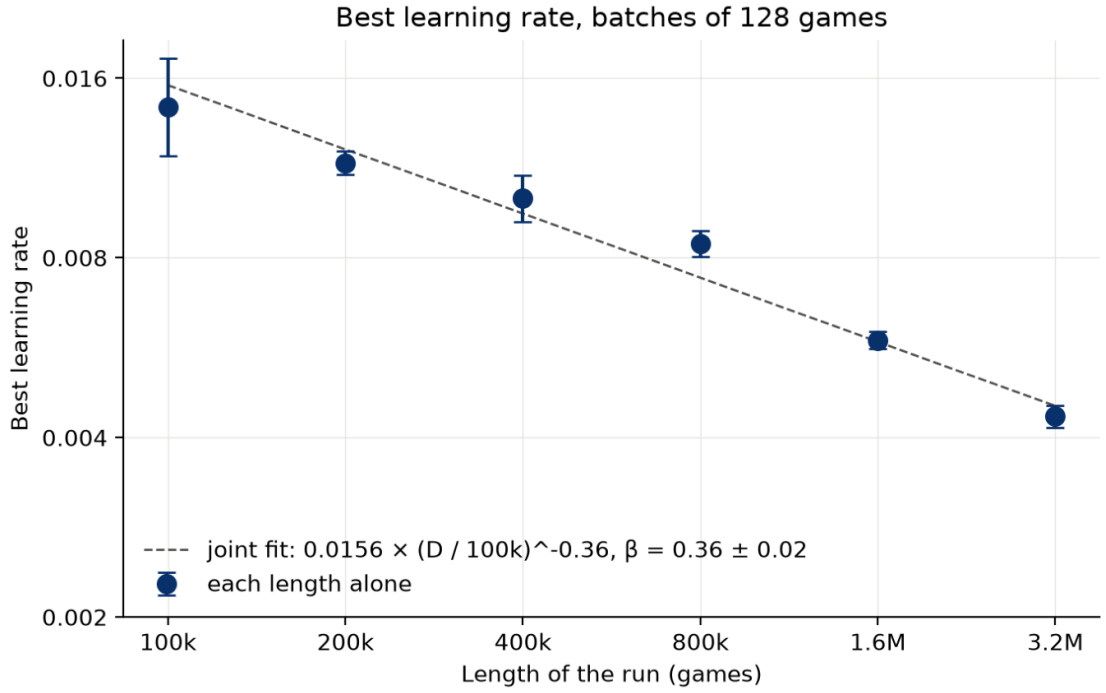


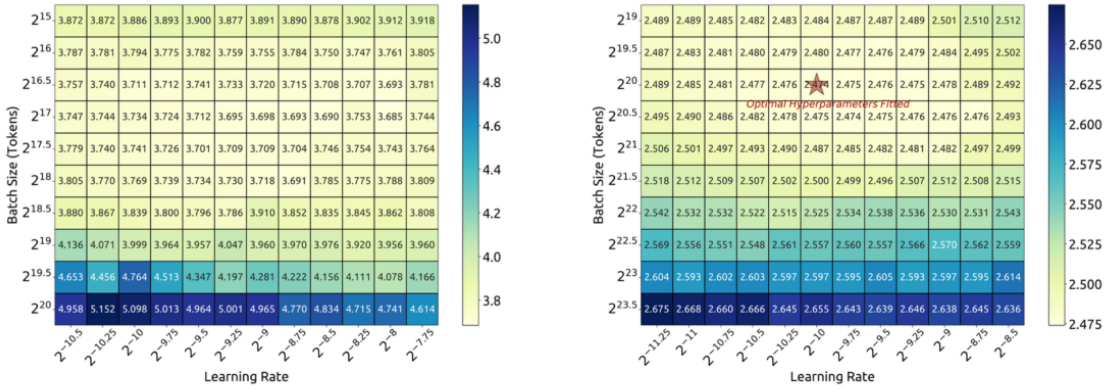
Figure 5.6: P1:64: log–log fit of optimal learning rate against training horizon in the chess experiment. The fit estimates a trend within the measured range; it is not a guarantee of safe extrapolation to an arbitrary dataset.

The later compute-scaling exercise uses a nearby operational rule, $0.0113(G/100{,}000)^{-0.357}$, rather than exactly the normalization in Equation (5.3). Preserve this distinction when reproducing the slides: an empirical optimum estimate and a selected training recipe need not have identical intercepts.

5.5.2 Unique data versus repeated exposures

Repeating a dataset for a few epochs might initially produce gains similar to using more data, but this remains a hypothesis for these experiments. No controlled comparison here establishes a fixed safe number of repetitions. Distinguish the number of *unique* tokens from the number of *training exposures*. Repeated tokens still consume compute and updates, but their information value can diminish. A horizon rule fitted on fresh data does not automatically predict the best recipe for repeated data.

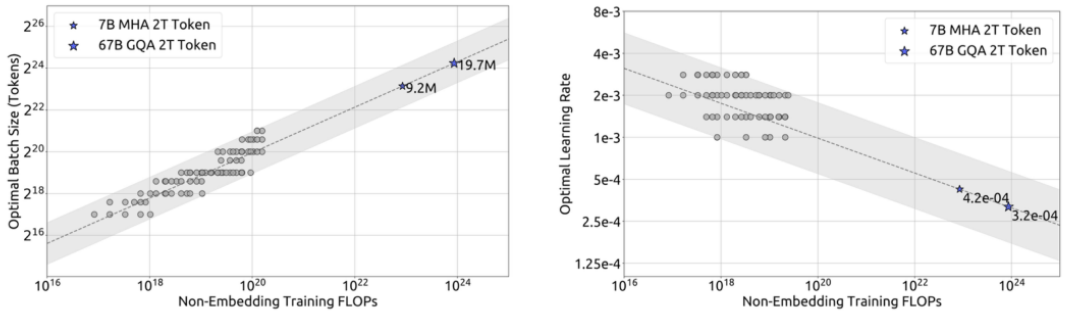
5.6 Connecting the small experiment to large-scale practice



(a) 1e17 FLOPs (177M FLOPs/token) (b) 1e20 FLOPs (2.94B FLOPs/token)

Figure 2 | Training loss w.r.t. batch size and learning rate with 1e17 and 1e20 FLOPs.

$$\eta_{\text{opt}} = 0.3118 \cdot C^{-0.1250}$$
$$B_{\text{opt}} = 0.2920 \cdot C^{0.3271} \quad (1)$$



(a) Batch size scaling curve (b) Learning rate scaling curve

Figure 5.7: P1:65: DeepSeek's joint exploration of learning rate, batch size, and compute. These empirical laws provide another example of tuning the recipe as scale changes.

The DeepSeek LLM report [8] fits optimal batch size and learning rate against compute. The fitted powers are approximately $B \propto C^{0.3271}$ and $\eta \propto C^{-0.1250}$. The numeric prefactors depend on the units and definitions used in that study. It would be meaningless to insert FLOPs into a fitted expression whose C was normalized differently.

What to retain. Tune learning rate and batch size together; measure seed variability; retune or transfer the learning rate when the horizon changes. A sensible small-scale optimum is the starting point for scaling, not the end of the problem.

Chapter 6

Transferring a recipe to wider models

Lecture slides: P1:67–73.

Retuning a large model by an exhaustive search is expensive. The goal is to discover a rule on a cheap model and transfer it to a wider one.

6.1 The empirical problem with standard parameterization

Keep four transformer layers and vary the embedding width d. With standard parameterization, the best learning rate decreases as width grows. Since the dominant parameter count scales approximately as Ld^2, a doubling of width roughly quadruples parameters at fixed depth.

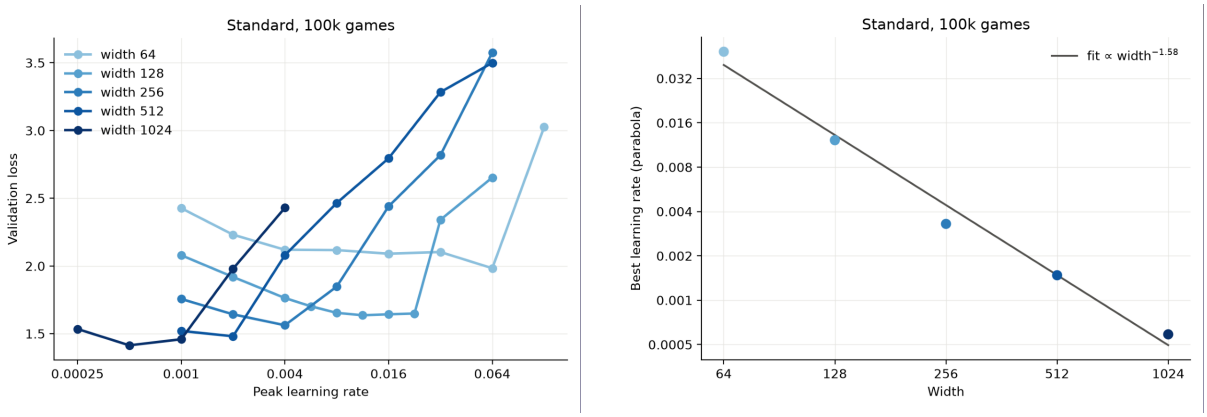


Figure 6.1: P1:67–68: wider models require smaller learning rates in this standard-parameterization experiment. The fitted exponent printed on P1:68 is about -1.58.

This small sweep supports a local empirical trend, not a precise universal law. P1:68 gives a fitted exponent near -1.58, while P1:70 gives a value near -1.34. Both support the qualitative trend. They do not justify reporting a uniquely measured exponent without qualification.

6.2 Why width changes the meaning of a parameter update

Consider one component of a linear layer:

$$y_i = \sum_{j=1}^{d} W_{ij} x_j, \qquad \Delta y_i = \sum_{j=1}^{d} \Delta W_{ij} x_j.$$

An Adam update can have a coordinate scale of order η, because its numerator is divided by an estimate of its root-mean-square magnitude. If update contributions add coherently, the resulting feature change can scale like $d\eta$. Keeping a feature change of order one then suggests $\eta \propto d^{-1}$.

This argument provides intuition rather than a complete proof. If terms were independent and centered, their sum would scale differently, like $\sqrt{d}$ in standard deviation. Learned updates are correlated with activations and gradients, so independence is generally the wrong assumption. A valid theory must track those correlations, initialization scales, forward activations, backward signals, and the output layer together.

6.3 What maximal update parameterization proposes

The Tensor Programs V paper [34] develops *maximal update parameterization*, written μP, to preserve nontrivial feature learning as width grows. Its practical payoff is often called μ*Transfer*: tune a base learning rate at small width, then reuse it with prescribed parameter-group scaling at larger width.

For width-only scaling with Adam, the operational intuition is:

- scale learning rates for hidden weight matrices inversely with the width multiplier;
- treat vector-like parameters, such as normalization gains, differently;
- use the appropriate embedding and readout conventions;
- keep initialization and forward-pass scaling consistent with the chosen parameterization.

Rescale weight decay for matrices to preserve the decay timescale. If a particular parameter group's learning rate becomes η/r, choosing its decay coefficient as $r\lambda$ preserves the product $\eta\lambda$. This is a statement about an AdamW implementation that multiplies decay by learning rate; optimizer conventions must be checked.

Further explanation. The rule "divide every learning rate by width" is not μP. Different tensors have different width-dependent dimensions. The reference implementation [22] records base and target shapes and provides a compatible readout layer, initialization, and optimizer handling. A correct implementation needs that full specification. The simplified recipe above is therefore specific to this model family.

6.4 How to validate transfer experimentally

Before an expensive run, compare several widths using the same *base* learning-rate grid. Under successful transfer, the curves can move downward as model capacity improves while their minima remain near the same horizontal coordinate.

Two different checks are useful. A *coordinate check* measures forward activations or feature changes across widths for a small number of steps. It catches scaling errors before a long run. A *transfer check* measures final loss across learning rates and widths. It tests the desired practical result. Passing one does not imply passing the other.

Keep the scope of the claim narrow. Changing depth, attention conventions, optimizer, batch size, sequence length, or data distribution can require additional rules or a new calibration. In particular, μP addresses width dependence; the horizon dependence established in the preceding chapter remains a separate issue.

What to retain. Learning rates depend on parameterization. Transferring small-model tuning to larger models requires compatible width scaling, initialization, readouts, and parameter groups.

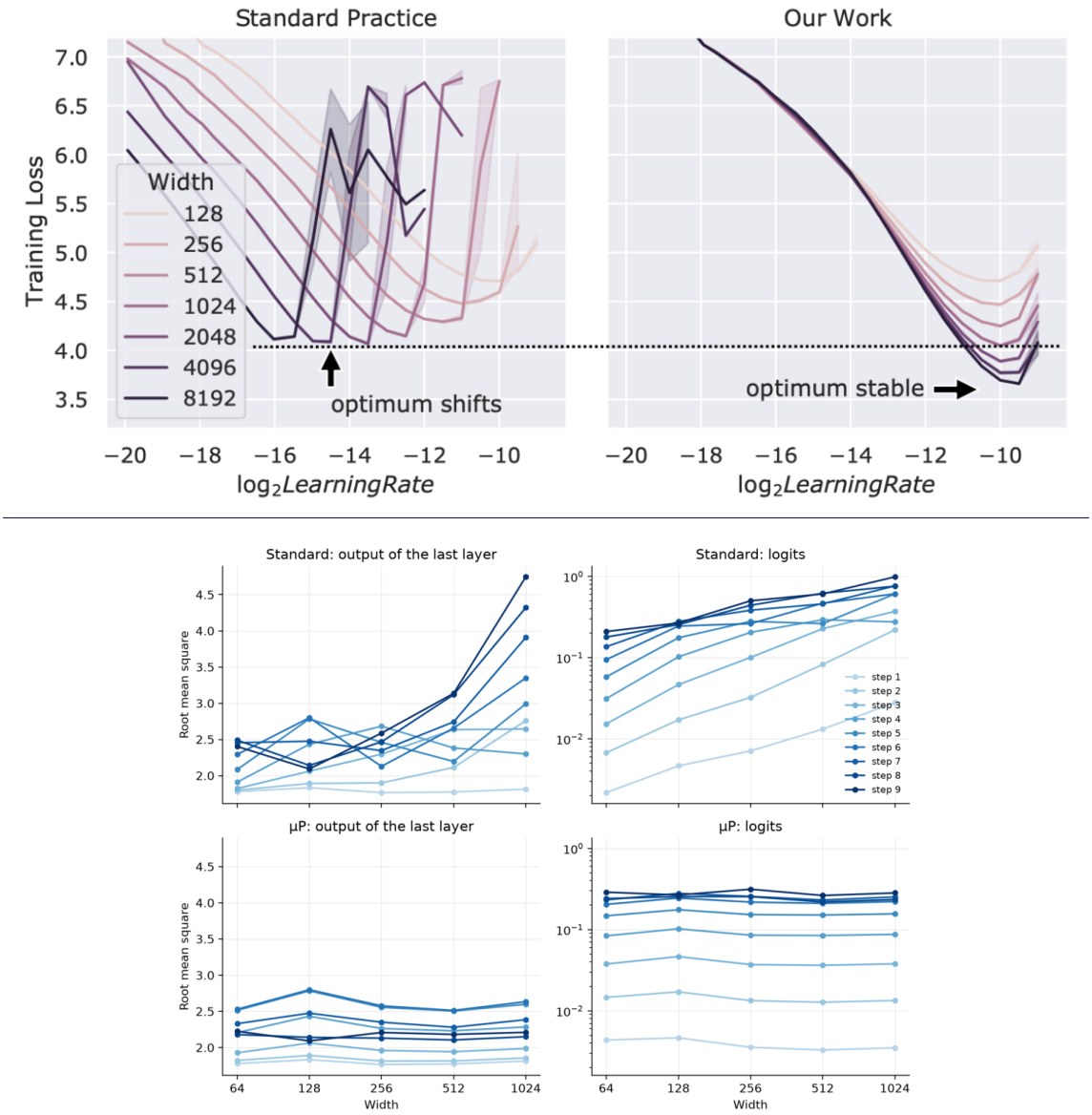


Figure 6.2: P1:71–72: the transfer idea and a coordinate check. Coordinate checks test whether activation scales remain controlled as width changes; they do not by themselves establish optimal validation loss.

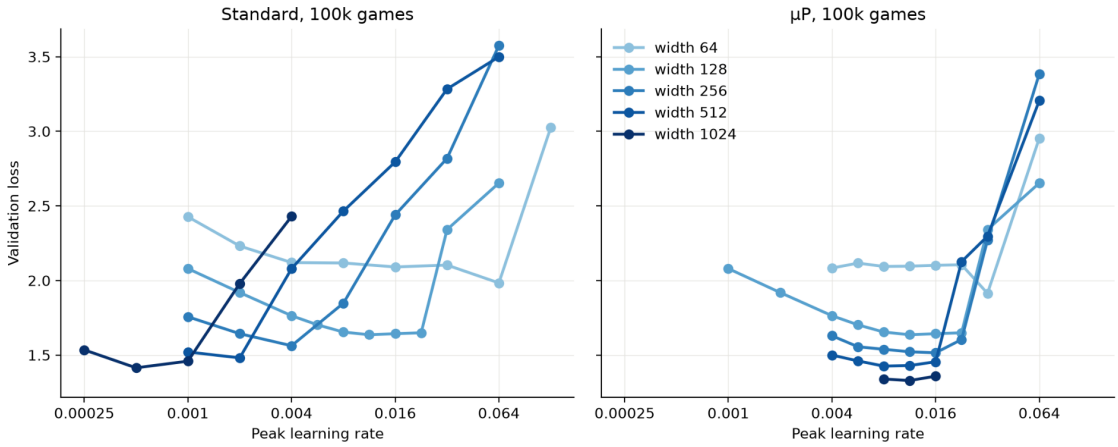


Figure 6.3: P1:73: chess experiments after width-dependent rescaling. The optimal base learning rates approximately align, making a small-model sweep useful for wider models.

Chapter 7

Compute-optimal scaling and a prediction tested

Lecture slides: P1:74–90.

7.1 What should we do with more compute?

If a training budget doubles, one can train the same model twice as long, double the model size for the same number of tokens, or increase both by intermediate factors. All three choices have approximately the same dense-model training cost $C = 6ND$. They need not produce the same loss.

The historical studies by Kaplan et al. [13] and Hoffmann et al. [11] investigate this allocation problem. The first finds a strong preference for increasing model size, approximately $N \propto C^{0.73}$ and $D \propto C^{0.27}$ in its setting. Chinchilla revises the balance toward approximately equal exponents. The distinction is about a *training procedure plus model family plus data distribution*, not an immutable property of transformers.

7.2 Chinchilla's approximately balanced rule

The Chinchilla study trains more than 400 models, spanning roughly 70 million to 16 billion parameters and 5 to 500 billion training tokens. Several estimation methods point toward scaling model size and data approximately together. Its celebrated practical summary is

$$N^*(C) \propto C^{1/2}, \qquad D^*(C) \propto C^{1/2}, \qquad D^* \approx 20N^*. \tag{7.1}$$

The resulting 70-billion-parameter Chinchilla model is trained on 1.4 trillion tokens. The conclusion is that many contemporary large models had used too few tokens relative to their size for their training budgets.

Training a fixed model for 30 rather than 20 tokens per parameter does not become harmful simply because it crosses 20. The statement is that, for a fixed *total training compute*, another allocation between model size and tokens may be more efficient. With fresh useful data and stable optimization, extra training generally continues to help that fixed model, although gains diminish. Data repetition, distribution shifts, and overfitting can complicate that expectation.

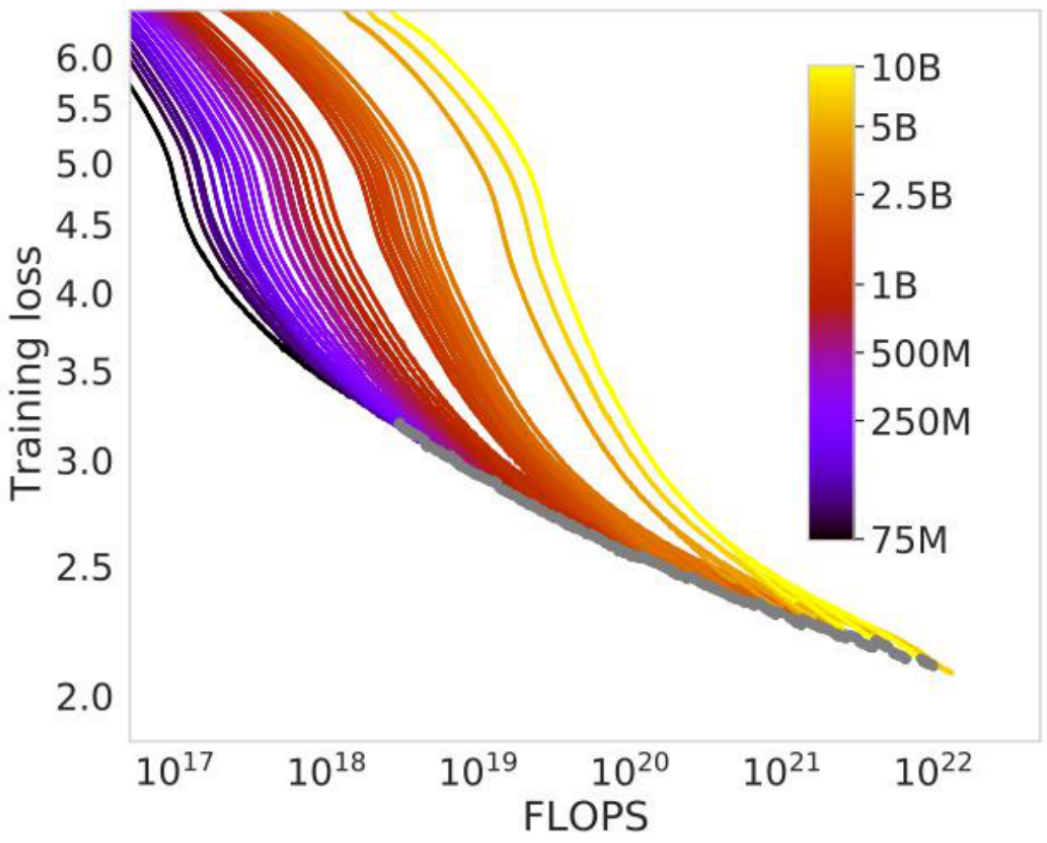


Figure 7.1: P1:76: training-loss trajectories and their efficient envelope. At a fixed compute value, the relevant comparison is vertical: which model and training duration produce the lowest loss?

7.2.1 A concrete allocation example

Suppose $C = 6 \times 10^{21}$ FLOPs and adopt $D = 20N$ as a planning approximation. Then

$$C = 120N^2, \quad N = \sqrt{C/120} \simeq 7.07 \times 10^9, \quad D \simeq 1.41 \times 10^{11}.$$

Four times the compute permits roughly twice the parameters and twice the tokens. These are allocation calculations, not a claim that this exact model will achieve a specified downstream score.

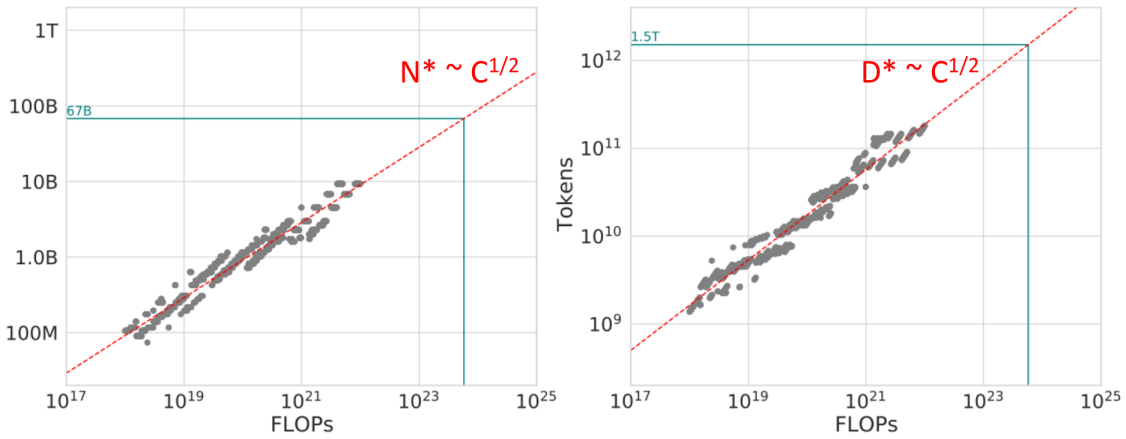
7.3 IsoFLOP curves and a loss surface

An *isoFLOP* comparison fixes C and varies N, setting $D = C/(6N)$. Very small models have many training tokens but insufficient capacity; very large models have capacity but too little training. A well-designed sweep often produces a U-shaped curve in log model size.

A common phenomenological model is

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \qquad A, B, \alpha, \beta > 0. \tag{7.2}$$

The first reducible term represents the limitation associated with model size; the second represents the limitation associated with training data. E is a fitted floor. Under an idealized correctly specified next-token model it is related to the entropy rate of the data source. In an empirical



Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Figure 7.2: P1:77–78: fitted compute-optimal scaling and the historical parameter/token comparison. The ratio of 20 is an approximate recipe from a particular study, not a universal stopping criterion.

finite-range fit, however, E can absorb model mismatch and fitting error. It is not a direct measurement of a universal "entropy of language." It also depends on tokenization and the evaluation distribution.

7.3.1 Deriving the optimal exponents

Substitute $D = C/(6N)$ into Equation (7.2):

$$\mathcal{L}(N; C) = E + AN^{-\alpha} + B\left(\frac{6N}{C}\right)^{\beta}.$$

At the optimum, differentiating with respect to N gives

$$\alpha A N^{-\alpha} = \beta B D^{-\beta}.$$

Hence

$$N^*(C) = \left(\frac{\alpha A}{\beta B\, 6^{\beta}}\right)^{1/(\alpha+\beta)} C^{\beta/(\alpha+\beta)}, \tag{7.3}$$

$$D^*(C) = \frac{C}{6N^*(C)} \propto C^{\alpha/(\alpha+\beta)}. \tag{7.4}$$

Equal compute exponents follow when $\alpha \approx \beta$. A constant token-to-parameter ratio follows only when the exponents are equal; its value also depends on A and B. The minimized reducible loss scales as $C^{-\alpha\beta/(\alpha+\beta)}$ in this idealized model.

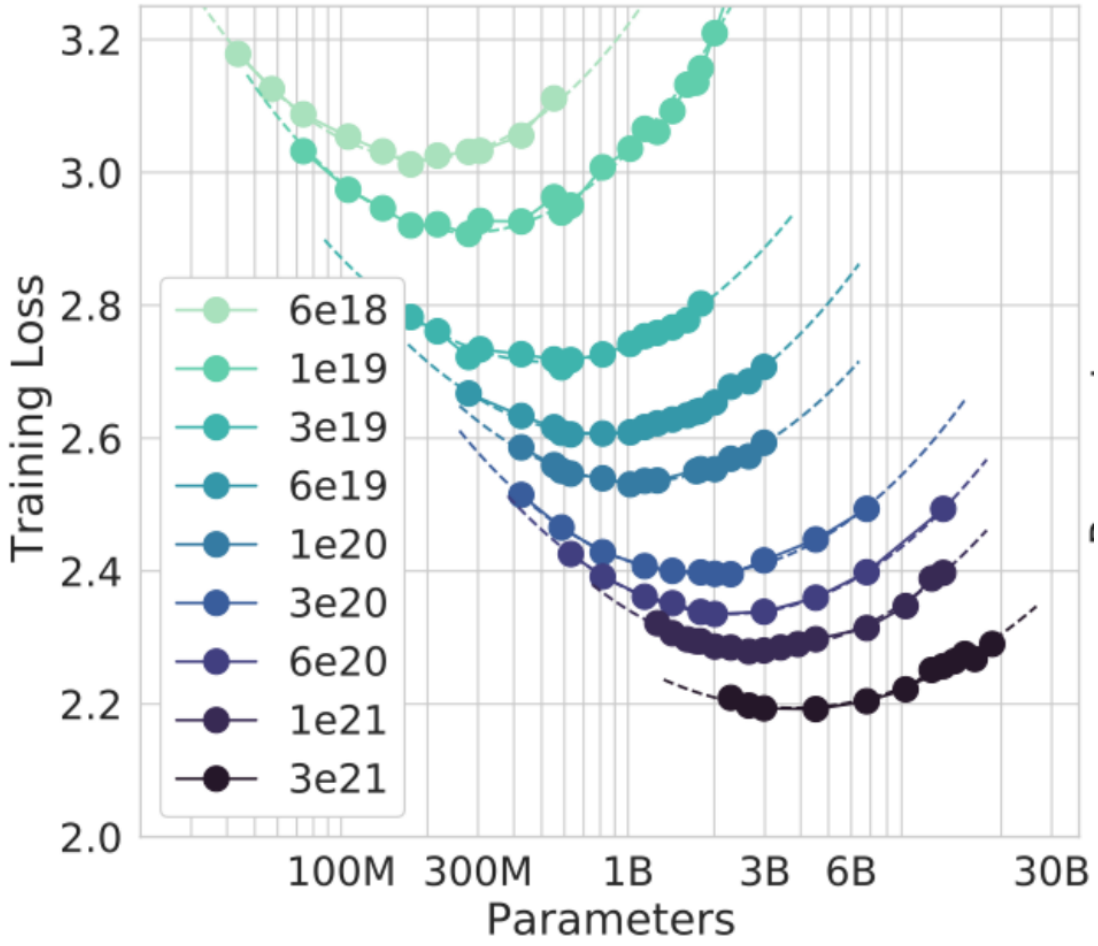


Figure 7.3: P1:79: isoFLOP curves. Each curve compares models trained with the same approximate compute. The minimum shifts toward larger models as the budget increases.

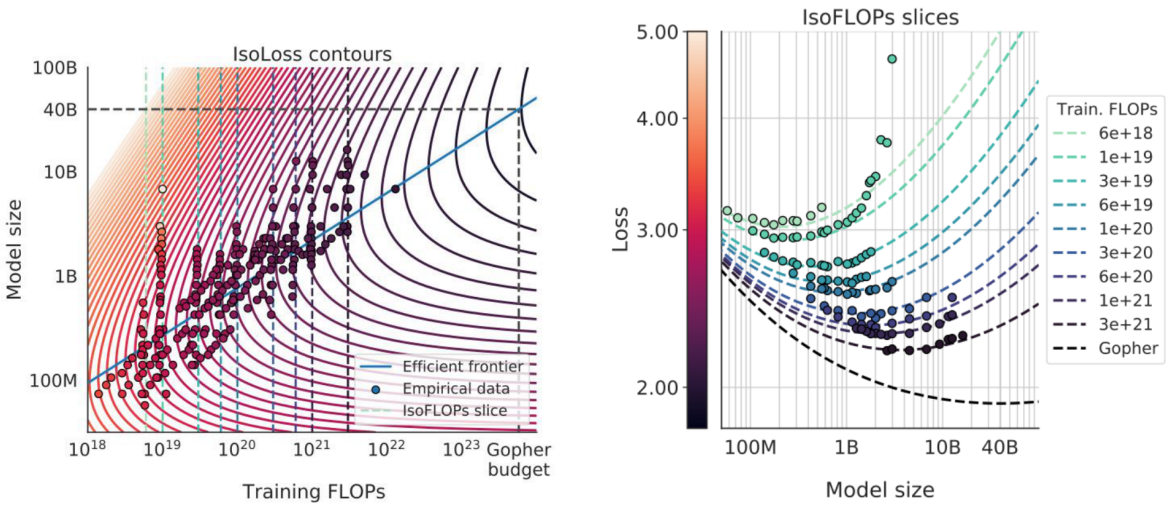


Figure 7.4: P1:80: two views of the fitted loss surface. Contours describe equally good parameter/data combinations; fixed-compute slices identify the best allocation at each budget.

7.4 Why careful tuning changes a scaling law

The discrepancy study by Porian et al. [27] revisits the Kaplan–Chinchilla disagreement with more than 900 runs. It identifies important effects from counting the output-head compute, choosing an appropriate warmup duration, and tuning optimizer parameters across scales. An apparent model-size penalty can partly be an optimization penalty.

Before a large scaling sweep, establish a usable batch size, a learning-rate versus horizon rule, and width transfer. Otherwise, larger or longer runs might be systematically mistuned, and a fit would confidently extrapolate that mistake.

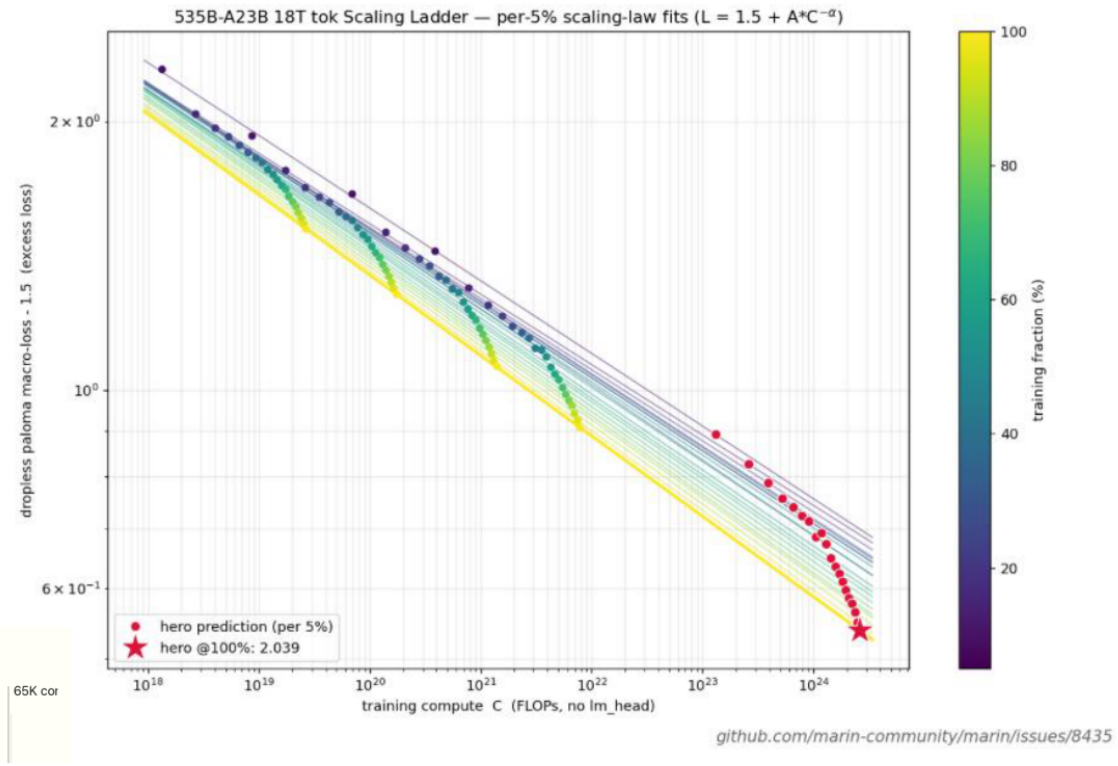


Figure 7.5: P1:81: Marin forecasts extrapolated from smaller runs. Forecasting beyond the fitted range tests the combined scaling model and training recipe.

The Marin report [17] describes preregistered predictions extending about 300 times beyond the fitted compute range. Its relevance here is methodological: declare the recipe and prediction before observing the expensive result. Accurate interpolation on existing measurements is a weaker test than a successful prospective prediction.

7.5 Training-optimal is not lifetime-cost-optimal

For a heavily used model, deployment compute can exceed pretraining compute. Minimizing combined training and deployment cost can therefore favor a smaller model trained for substantially more than the Chinchilla token ratio. "Overtraining" in this usage means training beyond the training-compute optimum for its size; it does not necessarily mean statistical overfitting.

Further explanation. A simplified lifetime-cost model is

$$C_{\text{total}} \approx 6ND + \kappa NQ,$$

where Q is the number of served tokens and κ summarizes inference cost per parameter per token. It omits attention-cache costs, batching, communication, and latency constraints. At a target quality, spending more training compute on a smaller model may save $\kappa \Delta N Q$ during serving. As Q grows, that saving becomes more important. There is consequently no deployment-independent optimal token-to-parameter ratio.

7.6 The chess scaling experiment

7.6.1 The experimental design

The small chess setting makes the full scaling methodology testable on accessible hardware. The reported initial sweep took roughly one night on the experimental setup; its runtime is hardware-dependent. The design has 18 runs across four approximate compute budgets:

$$C \in \{3 \times 10^{14},\ 10^{15},\ 3 \times 10^{15},\ 10^{16}\} \text{ FLOPs.}$$

Depth remains four, widths range approximately from 96 to 512, and batch size is 128. WSD supplies comparable cooldown endpoints. Width uses the transfer recipe, while the base learning rate follows

$$\eta_{\text{base}} = 0.0113 \left(\frac{G}{100{,}000} \right)^{-0.357}. \tag{7.5}$$

7.6.2 Fitting each fixed-budget minimum

A useful local approximation is a quadratic in $\log N$ near each minimum. It is an interpolation device, not a replacement for the full loss surface. A minimum at the boundary of the tested sizes is not well determined; the sweep should extend past it on both sides.

P1:86 also includes a learning-rate check. Multiplying the transferred rate by two improves some runs, while halving it often hurts. For example, at width 256 and about 390,000 games the displayed losses are 1.2865, 1.2556, and 1.2448 for factors 0.5, 1, and 2. Thus the transferred recipe is useful but not exactly optimal at every point.

7.6.3 The resulting allocation rule

Compute (FLOPs)	Parameters	Width	Games	Tokens/parameter
3×10^{14}	0.86 M	134	431,000	68
10^{15}	1.59 M	182	776,000	66
3×10^{15}	2.79 M	241	1,328,000	64
10^{16}	5.15 M	328	2,392,000	63

Table 7.1: Rounded fitted optima from P1:87. The parameter and game counts are interpolated recommendations, not necessarily individual runs in the sweep.

These scaling estimates use approximately 135 real tokens per game. The sequences are padded to 256 positions. If an implementation computes densely through all padding positions, the physical FLOPs are higher than the $6ND_{\text{real}}$ labels. The consistent interpretation is a real-token-based proxy budget, not a hardware measurement. This distinction affects absolute budgets, while a constant padding fraction need not change the fitted exponents.

C (6ND)	Width	N	Games	Tokens	Peak LR
3e14	96	0.45M	823k	111M	0.0053
3e14	128	0.80M	465k	63M	0.0065
3e14	192	1.78M	207k	28M	0.0087
3e14	256	3.16M	117k	16M	0.0107
1e15	96	0.45M	2.74M	371M	0.0035
1e15	128	0.80M	1.55M	210M	0.0042
1e15	192	1.78M	692k	93M	0.0057
1e15	256	3.16M	390k	53M	0.0070
1e15	384	7.11M	174k	23M	0.0093
3e15	128	0.80M	4.65M	629M	0.0029
3e15	192	1.78M	2.07M	280M	0.0038
3e15	256	3.16M	1.17M	158M	0.0047
3e15	384	7.11M	521k	70M	0.0063
3e15	512	12.62M	293k	40M	0.0077
1e16	192	1.78M	6.92M	935M	0.0025
1e16	256	3.16M	3.90M	527M	0.0031
1e16	384	7.11M	1.74M	235M	0.0041
1e16	512	12.62M	978k	132M	0.0050

Figure 7.6: P1:84: the actual run grid. Its purpose is to compare different parameter/data allocations within each budget after transferring the optimization recipe.

7.6.4 The loss fit and its limits

The formula visible on P1:88 is

$$\mathcal{L}(N, D) \simeq 0.66 + \frac{32}{N^{0.32}} + \frac{469}{D^{0.41}}. \tag{7.6}$$

Here N and D are counts, not values in millions. The supplied PDF contains no visible plot underneath this formula. The formula is therefore analyzed directly; no reconstructed plot is presented as an original slide figure.

The rounded exponents in Equation (7.6) imply, by Equation (7.4), $N^* \propto C^{0.41/0.73} \simeq C^{0.562}$ and $D^* \propto C^{0.438}$. These are not exactly the separately fitted isoFLOP exponents 0.51 and 0.49. Different fitting methods, limited data, and rounding can produce different estimates. The robust conclusion is an approximately balanced tradeoff, not numerical identity among all fits.

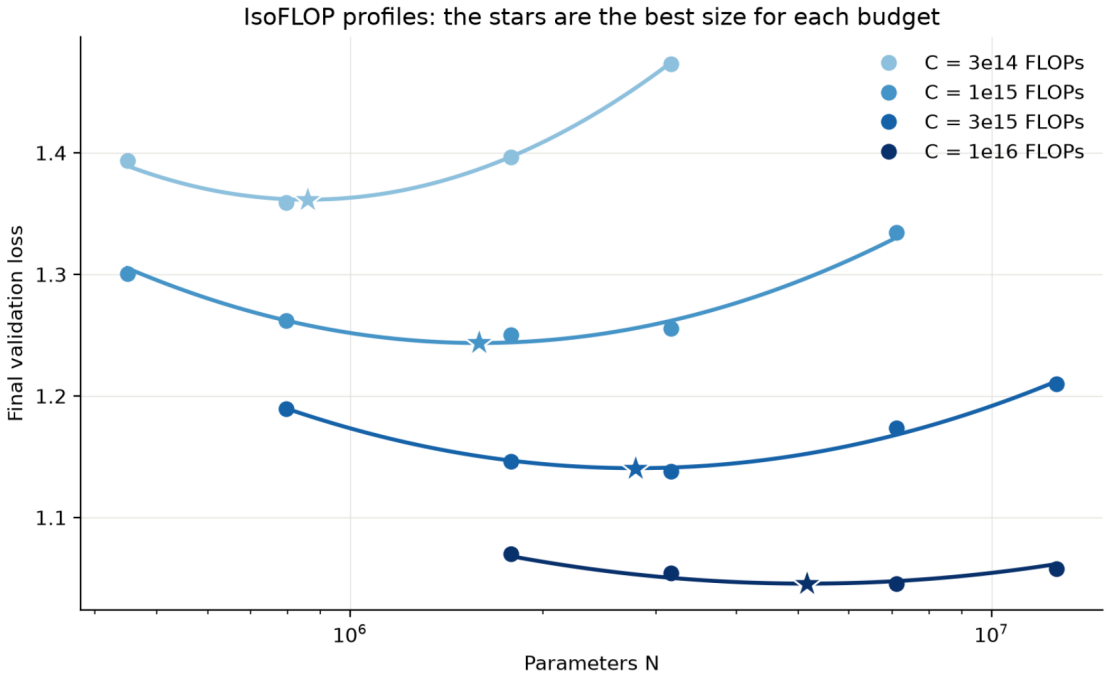


Figure 7.7: P1:85: chess isoFLOP curves and local fits. The curves reproduce the qualitative tradeoff found in language modeling, despite the very different token distribution.

7.6.5 The larger run: a prospective check

The larger validation run uses approximately 10^{17} FLOPs, width 576 (about 16 million parameters), and about 7.7 million games. An H100 accelerates this run. Most tuning experiments had used models below one million parameters.

The difference is about -0.012 nats per token: the observed loss is slightly better than predicted. This is persuasive evidence that the small experiments captured useful structure. It is not proof that the same exponent, token ratio, or error bound will hold for a new architecture or natural-language dataset. The printed prediction may also use unrounded fit coefficients; it should not be expected to match every calculation made from the rounded equation exactly.

What to retain. Scaling laws make expensive training a testable forecast. Their reliability depends on consistent token accounting, well-tuned comparison runs, uncertainty near the minima, and validation outside the fitting range.

Width	Games	Recipe learning rate	× 0.5	× 1	× 2
96	2.74M	0.0035	1.3389	**1.3006**	1.3027
128	1.55M	0.0042	1.2760	1.2620	**1.2512**
192	692k	0.0057	1.2789	1.2503	**1.2486**
256	390k	0.0070	1.2865	1.2556	**1.2448**

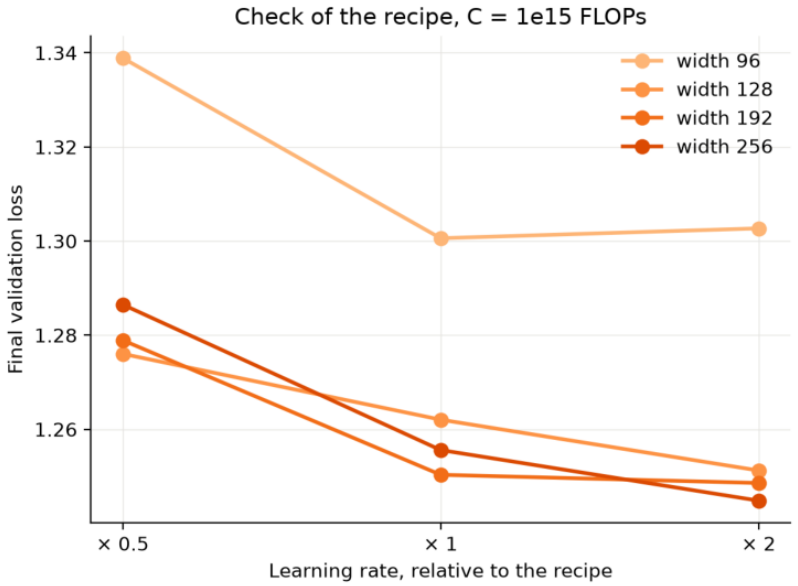


Figure 7.8: P1:86: sensitivity to a factor-of-two change in learning rate. Residual tuning error is one reason not to treat the fitted scaling coefficients as exact constants.

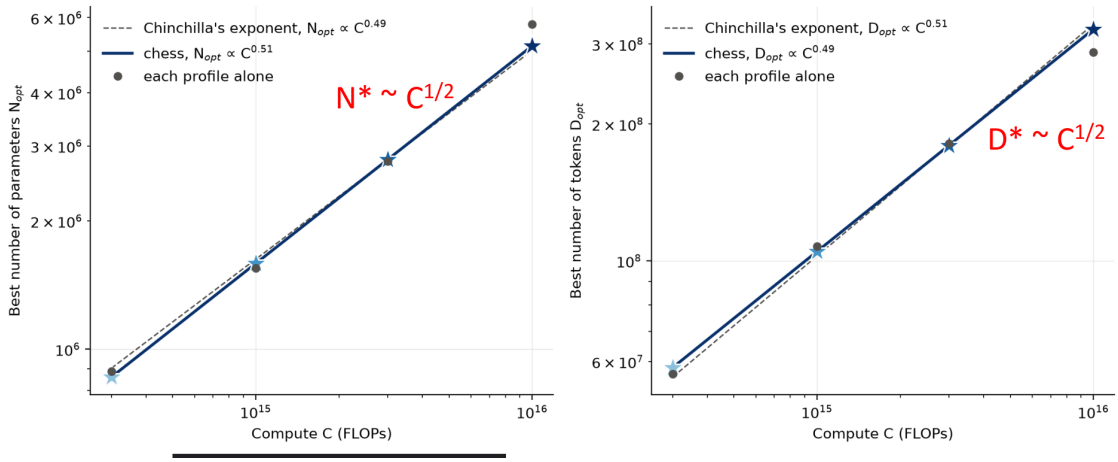


Figure 7.9: P1:87: fitted optimal size and token count for chess. The displayed exponents are about 0.51 and 0.49, and the optimum is around 65 real tokens per parameter in this setting.

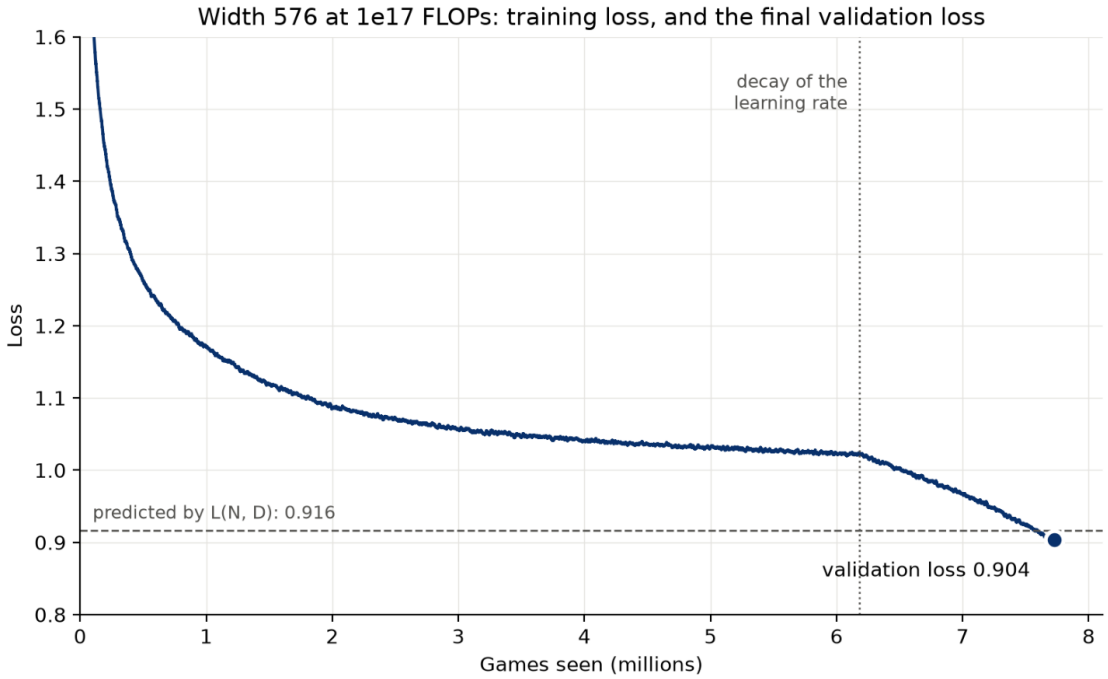


Figure 7.10: P1:90: the larger chess run and its forecast. The predicted loss is 0.916 and the measured validation loss is 0.904. The final WSD cooldown produces the visible late improvement.

Chapter 8

Selecting and preparing pretraining data

Lecture slides: P2:1–18.

8.1 The objective changes when the dataset changes

In the chess optimization experiments, a fixed validation distribution made loss an effective comparison metric. When choosing between datasets, the validation distribution itself can change. A corpus containing one repeated token is easy to predict and has low entropy, but is not a useful general-language training set.

Formally, cross-entropy is

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q).$$

Changing the evaluation distribution p changes its intrinsic entropy $H(p)$. Thus a lower loss on a different validation set need not imply a better model. Keep common evaluation sets and tokenization when comparing recipes, and supplement loss with downstream tasks.

A base model can be evaluated without instruction tuning by comparing the likelihoods of candidate answers. Given a question q and candidate answers a_j, score each continuation by

$$s_j = \log p_\theta(a_j \mid q) = \sum_t \log p_\theta(a_{j,t} \mid q, a_{j,<t}), \qquad \hat{j} = \arg\max_j s_j.$$

The model need not obey a conversational instruction: it only needs to assign probabilities to continuations. The evaluation protocol must specify the prompt, whether answer length is normalized, and whether calibration against an answer-only baseline is used. These choices can change the ranking, so use the same protocol for every dataset ablation.

8.2 From a web crawl to a training corpus

Common Crawl provides large web archives [6]; it is a source pool, not a ready-made high-quality language-model dataset. The same page may appear in multiple crawl snapshots, and the raw material contains navigation, advertising, boilerplate, malformed pages, spam, and duplicates.

FineWeb [26] is the main data-engineering reference in this section. Its experiments motivate careful extraction, filtering, and deduplication rather than assuming that more raw bytes automatically improve training. The published corpus contains about 15 trillion tokens from 96 Common Crawl snapshots. FineWeb-Edu is a selected educational subset built using model-assisted scoring.

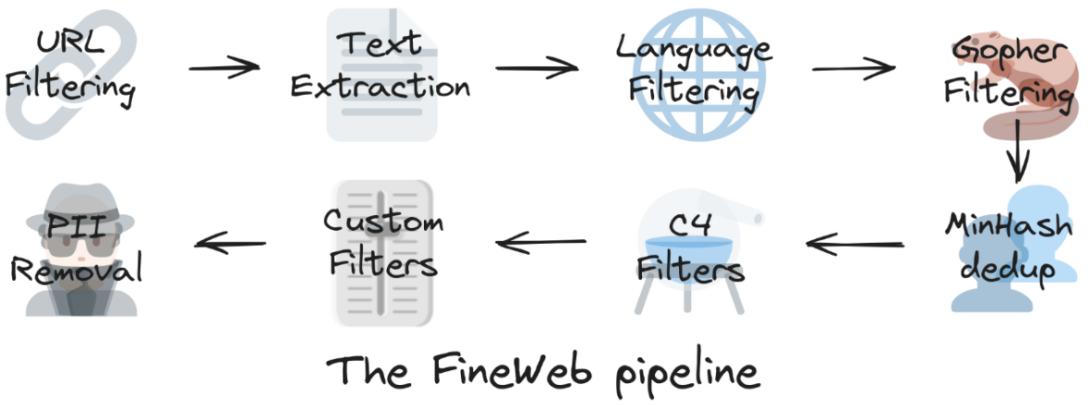


Figure 8.1: P2:4: the FineWeb preparation pipeline. Each stage changes both the quality and the composition of the remaining corpus.

Common Crawl October 2026 Crawl Archive (CC-MAIN-2026-43)

The October 2026 crawl archive contains 2.16 billion pages, see the announcement for details.

Data Size and File Listings

Data Type	File List	#Files	Total Size Compressed (TiB)
Segments	segment.paths.gz	100	
WARC	warc.paths.gz	100000	86.19
WAT	wat.paths.gz	100000	14.06
WET	wet.paths.gz	100000	5.85
Robots.txt files	robotstxt.paths.gz	100000	0.15
Non-200 responses	non200responses.paths.gz	100000	2.72
URL index files	cc-index.paths.gz	302	0.23
Columnar URL index files	cc-index-table.paths.gz	900	0.20

Figure 8.2: P2:6: the crawl download interface and the very different sizes of available representations. The displayed snapshot is a historical example, not a current download-size guarantee.

8.3 Content filtering and text extraction

8.3.1 Filter unwanted sources

URL blocklists, spam indicators, and content classifiers can remove sources that do not fit the intended corpus. These are policy and distribution choices as well as quality choices. A filter can preferentially remove a language variety, domain, or writing style. Inspect samples of both accepted and rejected material instead of judging a filter only by how much data it removes.

8.3.2 WARC and WET are different starting points

The WARC representation contains archived web responses, including HTML. WET contains extracted plain text. For the same page, WET looks more readable, but its generic extraction may retain irrelevant material or lose useful structure. Starting from WARC permits a purpose-built extraction pipeline.

Trafilatura [2] extracts main text and can preserve useful structure while suppressing page

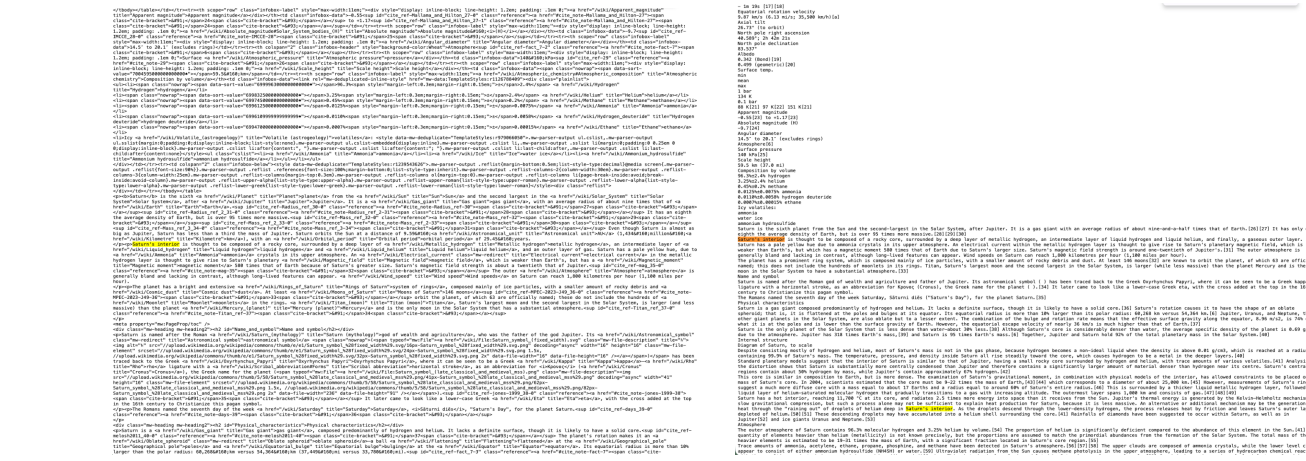


Figure 8.3: P2:7–8: the same web content as raw HTML and extracted text. Visual cleanliness alone does not establish that the useful main content has been recovered correctly.

boilerplate. Extraction is an upstream modeling decision: no amount of optimizer tuning can recover passages discarded before tokenization.

8.3.3 Simple quality heuristics

The Gopher data discussion [28] illustrates filters on document length, mean word length, symbol fractions, repeated lines or paragraphs, and excessive bullet-point structure. These are cheap indicators of malformed or repetitive content. They are not definitions of intellectual quality. A table, source-code file, mathematical derivation, or short poem may violate a web-prose heuristic while remaining useful data.

For a staged pipeline, log how many documents and tokens each filter removes and sample its decisions. Apply language-aware thresholds where needed. A filter tuned for English prose should not be assumed suitable for code or another writing system.

8.4 Deduplication: exact matches and near matches

8.4.1 Exact deduplication

Comparing every pair of documents directly is prohibitively expensive. A hash maps each normalized document to a compact signature. Grouping identical hashes finds likely exact duplicates efficiently. A sufficiently large hash makes accidental collisions rare; an exact comparison can resolve them if needed. Normalization choices matter: deleting all punctuation can incorrectly merge distinct code or mathematical expressions.

Exact matching misses a syndicated article with a different header or cookie banner. Near-duplicate detection instead compares the underlying content approximately.

8.4.2 MinHash, step by step

Split a tokenized document into overlapping shingles, such as five-word sequences. The sequence $w_1, \ldots, w_n$ gives shingles $(w_1, \ldots, w_5)$, $(w_2, \ldots, w_6)$, and so on. Represent the document by the *set* of its shingles, S. For a hash function h, define

$$m_h(S) = \min_{s \in S} h(s).$$

Two nearly identical documents share many shingles, so they often share this minimum. But a single changed shingle may happen to hold the minimum; one signature is noisy.

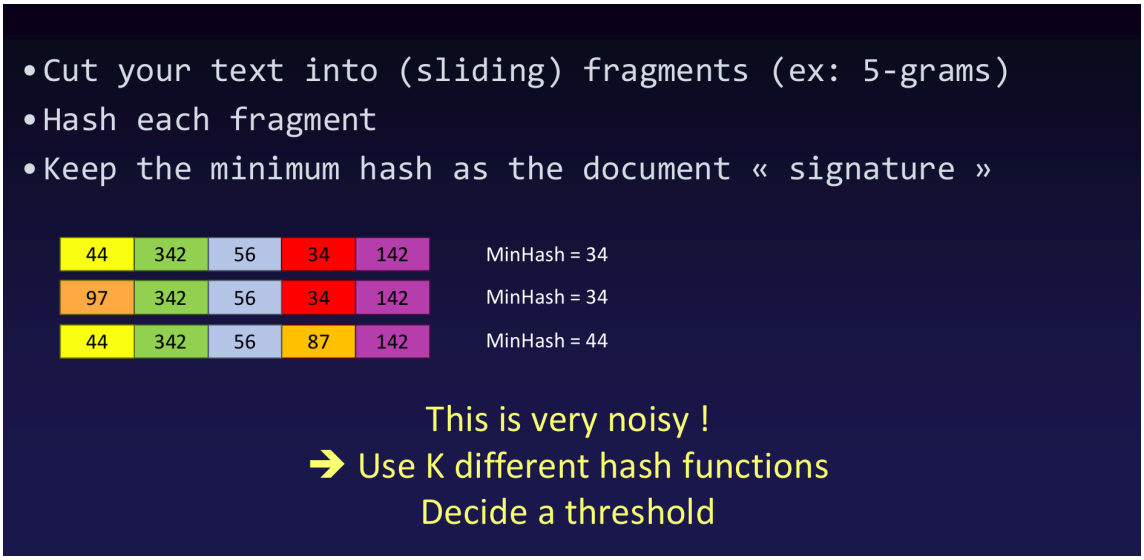


Figure 8.4: P2:12: a numerical MinHash example. A local edit often preserves the minimum, but can change it; multiple independent signatures reduce this uncertainty.

Further explanation. For an ideal random ordering of shingles, the probability of equal minima is exactly the Jaccard similarity:

$$\Pr[m_h(S) = m_h(T)] = J(S,T) = \frac{|S \cap T|}{|S \cup T|}. \tag{8.1}$$

The smallest element of $S \cup T$ is equally likely to be any element of that union. The two minima agree precisely when that element lies in the intersection. With K independent hashes, the fraction of agreeing minima estimates J, with standard error approximately $\sqrt{J(1-J)/K}$. Real hash families approximate this idealization.

Use multiple hash functions to estimate similarity, then apply a threshold to identify near duplicates. At web scale, comparing every pair of signatures would still be expensive. Locality-sensitive hashing groups signatures into b bands of r entries and retrieves pairs that match an entire band. Under the idealized independence model, a pair of similarity J becomes a candidate with probability

$$P_{\text{candidate}}(J) = 1 - (1 - J^r)^b. \tag{8.2}$$

FineWeb uses 112 signatures arranged as 14 bands of 8. This creates a smooth probability curve, not an exact similarity cutoff. Candidate grouping and the subsequent choice of which documents to retain are separate decisions.

Deduplication can change the domain mixture and remove useful repetition as well as contamination. The FineWeb ablations show why aggressive global deduplication cannot simply be assumed better than all alternatives. Evaluate the downstream effect of the actual retained corpus.

8.5 Personal data and benchmark contamination

Data preparation should account for email addresses, IP addresses, crawl exclusions, and benchmark canary strings. Redaction can reduce obvious personal identifiers but is not a

guarantee that all personal information has been removed. Conversely, pattern-based filters can alter harmless examples such as programming tutorials.

A benchmark canary is a distinctive string inserted into benchmark material so corpus builders can detect it. Finding one provides strong evidence that benchmark content entered the source pool. Remove or quarantine the associated content according to the evaluation protocol. Absence of the string does not prove absence of contamination: copies, translations, reformatted questions, or answer discussions may omit it.

For rigorous evaluation, preserve held-out sets, track provenance, and test for overlap beyond exact canary matching. Crawl permissions and benchmark notices are additional provenance and exclusion signals to preserve during collection.

8.6 Model-based filtering and educational quality

A classifier can assign each document a score for inclusion. The key question is where its labels come from. Two approaches are available: learn to distinguish reference sources such as Wikipedia or well-rated explanatory forum posts; or ask a language model to grade a sample, then train a cheaper classifier to imitate those judgments across the full corpus.

This second procedure is a form of distillation. The expensive teacher supplies labels; the small classifier supplies scale. The score reflects the teacher, rubric, and sampled documents. It can reward a recognizable explanatory style while missing factual errors or excluding useful material that looks different. Check held-out human judgments and downstream ablations, not only agreement with the teacher.

FineWeb-Edu applies this idea to educational value [26]. The methodological lesson is to separate generating labels, fitting the scoring model, choosing the acceptance threshold, and testing the resulting training mixture. Each stage can introduce a different bias.

8.7 Rephrasing and synthetic pretraining text

Filtering discards data; rewriting tries to improve or diversify it. A generator can transform a web passage into an explanation, question-and-answer text, structured summary, or another format. The result must be assessed for factual preservation, coverage, diversity, and computational cost. Synthetic text can reproduce the generator's errors and style biases.

Repeatedly replacing real data with model-generated samples can cause *model collapse*. Shumailov et al. [30], an additional reference, analyze how recursive generation can progressively lose the tails of the original distribution. This warning should not be strengthened into a universal theorem that every synthetic-data student must perform worse than its generator. Source-grounded rewriting, filtering, verification, and mixing with retained real data are materially different experimental settings.

The FinePhrase study [24], cited on P2:15, compares prompt design, generator size, and source material. In its tested settings, structured formats such as tables, mathematical exposition, FAQs, and tutorials can be effective; larger generators are not automatically worth their extra cost. Its released synthetic corpus is about 486 billion tokens. The important proposal is systematic comparison of the *generation recipe*, not a claim that any paraphrase improves any corpus.

Nemotron-CC [31] combines classifier-based selection and synthetic rewriting at much larger scale. It reports a 6.3-trillion-token corpus comprising about 4.4 trillion real and 1.9 trillion synthetic tokens. It treats aggressive heuristic removal as a quality–quantity tradeoff rather than an unquestionable preprocessing default. Both papers are useful examples of testing whether generated data actually improves a separately trained model.

8.8 Mixtures, midtraining, and context extension

8.8.1 A data pool is not the training mixture

Let source i contain U_i available tokens, and let its chosen sampling share be p_i. A run of D tokens consumes approximately $p_i D$ tokens from that source. Its effective number of exposures is

$$e_i = \frac{p_i D}{U_i}.$$

The source can be downsampled ($e_i < 1$) or repeated ($e_i > 1$). Thus "the dataset has nine trillion tokens" does not determine what a six-trillion-token run sees.

Source	Type	9T Pool		6T Mix		150B Mix	
		Tokens	Docs	Tokens	Docs	Tokens	Docs
Common Crawl	Web pages	8.14T	9.67B	4.51T (76.1%)	3.15B	121B (76.9%)	84.5M
olmOCR science PDFs	Academic documents	972B	101M	805B (13.6%)	83.8M	19.9B (12.6%)	2.25M
Stack-Edu (Rebalanced)	GitHub code	137B	167M	409B (6.89%)	526M	11.1B (7.06%)	14.3M
arXiv	Papers with LaTeX	21.4B	3.95M	50.8B (0.86%)	9.10M	1.29B (0.82%)	247K
FineMath 3+	Math web pages	34.1B	21.4M	152B (2.56%)	95.5M	4.10B (2.60%)	2.57M
Wikipedia & Wikibooks	Encyclopedic	3.69B	6.67M	2.51B (0.04%)	4.24M	64.6M (0.04%)	119K
Total		**9.31T**	**9.97B**	**5.93T (100%)**	**3.87B**	**157B (100%)**	**104M**

Table 4 Composition of Dolma 3 Mix including our 9T pool of data, the 6T mix we used for final model training, and the 150B mix we used for experimentation.

Figure 8.5: P2:16: the Olmo 3 source pool and selected mixtures. For example, code and mathematics are upweighted relative to their raw availability. The table distinguishes token counts from document counts.

To compare mixtures, use small controlled training runs with the same token and compute budgets, then evaluate domain-specific capabilities as well as an aggregate score. A change that improves mathematics while degrading multilingual coverage may or may not serve the intended model.

8.8.2 Midtraining

Midtraining is a late pretraining phase that shifts the data mix toward valuable domains while retaining the language-model objective. This is distinct from instruction tuning or preference optimization. A smaller high-quality pool can have substantial influence late in training, but its repetitions and interactions with the learning-rate schedule must be monitored.

8.8.3 Long-context training

Increasing the supported context length requires more than changing a maximum-length field. Positional encoding, attention memory, sequence packing, and the availability of coherent long documents all matter. Randomly concatenated short documents do not automatically teach long-range reasoning over a single document.

Olmo 3 [25] illustrates staged mixture design and a dedicated context-extension phase. The short-document share helps preserve existing capabilities, while long documents supply training signal at larger distances. Evaluate retrieval, multi-part reasoning, and ordinary short-context tasks after extension; successful processing of a long input is not itself evidence of effective use of distant information.

What to retain. Data preparation is a sequence of experimentally testable modeling choices.

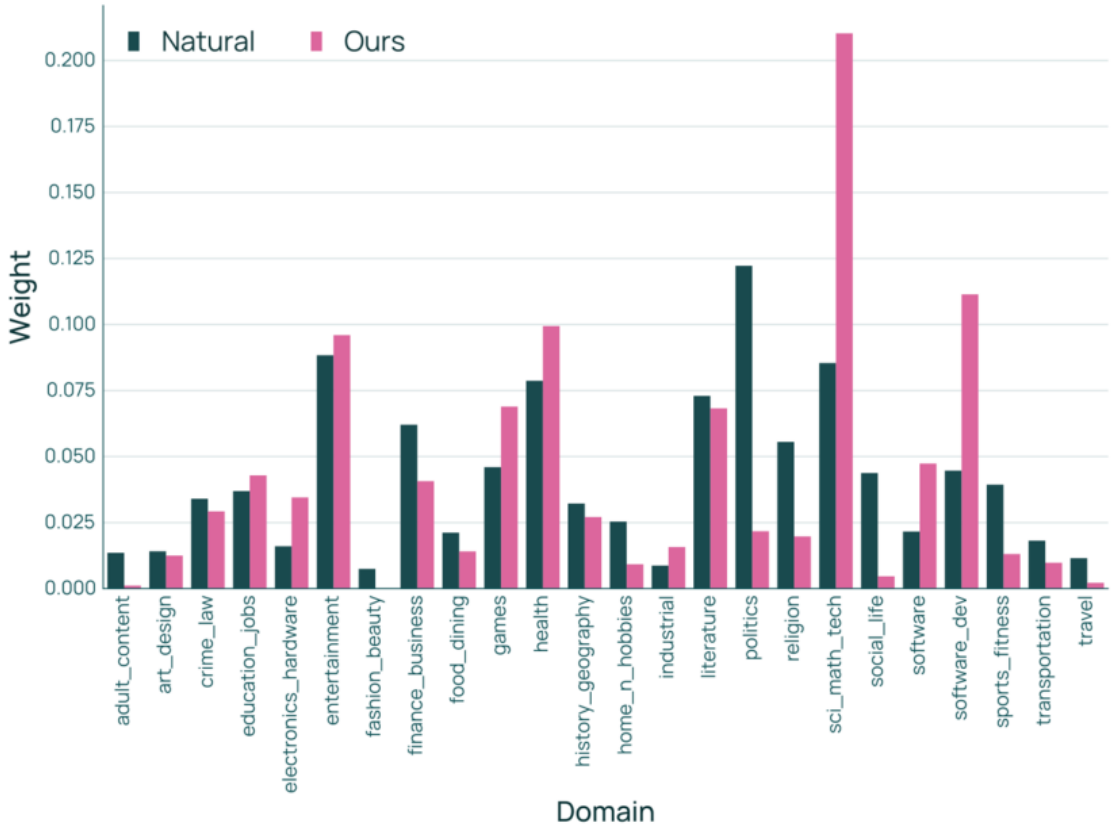


Figure 8.6: P2:16: the mixture visualization accompanying the table. Mixture proportions are part of the training recipe, not a passive consequence of collecting data.

Compare mixtures on common evaluations, distinguish source-pool size from sampled exposures, and treat filtering and synthetic rewriting as hypotheses rather than automatic improvements.

Type	Source	2T Pool		100B Mix	
		Tokens	Docs	Tokens	Docs
Math (synth)	TinyMATH Mind**	899M	1.42M	898M (0.9%)	1.52M
Math (synth)	TinyMATH PoT**	241M	729K	241M (0.24%)	758K
Math (synth)	CraneMath*	5.62B	6.55M	5.62B (5.63%)	7.24M
Math (synth)	MegaMatt*	3.88B	6.79M	1.73B (1.73%)	3.23M
Math (synth)	Dolmino Math^^	10.7B	21M	10.7B (10.7%)	22.3M
Code	StackEdu (FIM)^	21.4B	32M	10.0B (10.0%)	16.2M
Python (synth)	CraneCode*	18.8B	19.7M	10.0B (10.0%)	11.7M
QA (synth)	Reddit To Flashcards**	21.6B	370M	5.90B (5.9%)	101M
QA (synth)	Wiki To RCQA**	4.22B	22.3M	3.0B (3.0%)	16.3M
QA (synth)	Nemotron Synth QA^	487B	972M	5.0B (5.0%)	10.6M
Thinking (synth)	Math Meta-Reasoning**	1.05B	984K	381M (0.38%)	401K
Thinking (synth)	Code Meta-Reasoning**	1.27B	910K	459M (0.46%)	398K
Thinking (synth)	Program-Verifiable**	438M	384K	159M (0.16%)	158K
Thinking (synth)	OMR Rewrite FullThoughts^	850M	291K	850M (0.85%)	394K
Thinking (synth)	QWQ Reasoning Traces^	4.77B	438K	1.87B (1.87%)	401K
Thinking (synth)	General Reasoning Mix^	2.48B	668K	1.87B (1.87%)	732K
Thinking (synth)	Gemini Reasoning Traces^	246M	55.2K	246M (0.25%))	85.1K
Thinking (synth)	Llama Nemotron Reasoning Traces^	20.9B	3.91M	1.25B (1.25%)	368K
Thinking (synth)	OpenThoughts2 Reasoning Traces^	5.6B	1.11M	1.25B (1.25%)	402K
Instruction (synth)	Tulu 3 SFT^^	1.61B	1.95M	1.1B (1.1%)	1.45M
Instruction (synth)	Dolmino 1 Flan^^	16.8B	56.9M	5.0B (5.0%)	14.8M
PDFs	olmOCR science PDFs (HQ subset)^	240B	28.7M	4.99B (5.0%)	1.20M
Web pages	STEM-Heavy Crawl^	5.21B	5.16M	4.99B (5.0%)	5.53M
Web pages	Common Crawl (HQ subset)^	1.32T	965M	22.4B (22.5%)	18.3M
Total		**2.19T**	**2.52B**	**99.95B (100%)**	**236M**

Table 5 Composition of the midtraining data (Dolma 3 Dolmino Mix). Here we show the full composition of the midtraining data mix. **=newly-introduced synthetic dataset. *=novel recreation of existing data. ^^=reuse of previously-introduced data. ^=filtering or light transformation of existing external data.

Figure 8.7: P2:17: the Olmo 3 midtraining mixture. The change in proportions is deliberate; it should be evaluated as a separate training stage.

Source	Length bucket	600B Pool		50B Mix	
		Tokens	Docs	Tokens	Docs
olmOCR PDFs	8K-16K	144B (22.5%)	12.7M	2.27B (4.55%)	235K
olmOCR PDFs	16K-32K	115B (18.0%)	5.06M	1.85B (3.70%)	110K
olmOCR PDFs	32K-64K	106B (16.6%)	2.30M	4.81B (9.63%)	177K
olmOCR PDFs	64K-128K	96.0B (15.0%)	1.05M	–	–
olmOCR PDFs	128K-256K	60.8B (9.5%)	342K	–	–
olmOCR PDFs	256K-512K	35.1B (5.49%)	97.1K	–	–
olmOCR PDFs	512K-1M	21.5B (3.36%)	30.2K	–	–
olmOCR PDFs	1M+	26.9B (4.21%)	12.2K	–	–
olmOCR PDFs + synth **CWE**	32K-64K	8.77B (1.37%)	189K	1.94B (3.88%)	71.3K
olmOCR PDFs + synth **REX**	32K-64K	24.1B (3.77%)	492K	6.08B (12.2%)	217K
Midtraining data mix	Variable	–	–	33.0B (66.1%)	79.2M
Total		**639B**	**22.3M**	**50.0B (100%)**	**80.0M**

Figure 8.8: P2:18: Olmo 3's long-context mixture, extending the context from 8K to 64K. The selected 50-billion-token mixture retains about two-thirds shorter midtraining material alongside long-document sources.

Chapter 9

Monitoring a run and investigating instability

Lecture slides: P1:48 and P2:19–23.

9.1 Read several signals together

A scalar loss curve is an incomplete description of a training run. Plot loss, learning rate, and gradient norm on aligned horizontal axes to distinguish a scheduled change, an isolated difficult batch, and an instability that changes the subsequent trajectory.

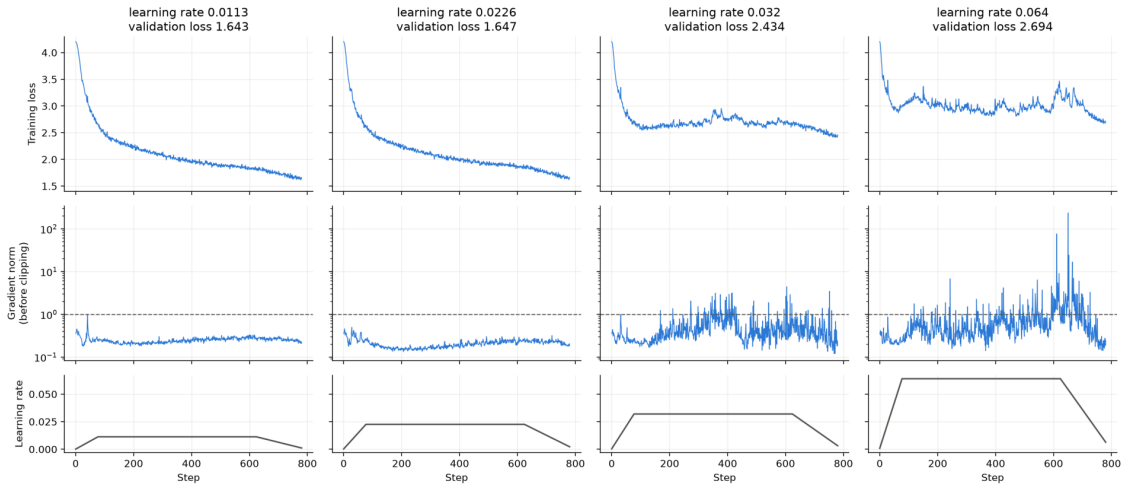


Figure 9.1: P2:20, revisiting P1:48: the anatomy of chess runs at different learning rates. Compare the timing of loss changes with gradient behavior and the schedule instead of interpreting each plot independently.

A useful record includes training loss, fixed-set validation loss, learning rate, pre-clipping gradient norm, the fraction of steps clipped, parameter and update norms, throughput, memory, and the data source of each batch. For attention instability, also inspect query/key magnitudes and attention-logit extremes. For mixed precision, record non-finite values and any skipped optimizer updates. These measurements answer different questions and should not be reduced to one dashboard number.

Training loss is a noisy batch measurement; validation loss is a sample estimate on a held-out distribution. A temporary training spike that immediately recovers may have little effect on the

latter. Conversely, a run can remain numerically finite while becoming systematically worse than a healthy continuation.

9.2 A rising gradient norm need not mean divergence

The chess plots show gradient-norm changes near the final decay. The paper by Defazio [10] studies a mechanism involving weight decay and normalized layers. For a scale-invariant function, changing a weight vector's magnitude need not change its output, while its gradient scale can change inversely. Shrinking weights can therefore increase the raw gradient norm without a corresponding deterioration in predictions.

The paper proposes an AdamC modification that couples the decay coefficient to the current learning rate in its setting: $\lambda_t = \lambda\eta_t/\eta_{\max}$. With the AdamW convention, the shrinkage term then scales as $\eta_t^2\lambda/\eta_{\max}$. This is a proposed intervention for the studied mechanism, not a reason to adopt it indiscriminately or to ignore genuinely unstable gradients.

Further explanation. A normalized gradient magnitude, a parameter-update ratio such as $\|\Delta\theta\| \,/\, \|\theta\|$, and validation loss can give a more meaningful picture than raw gradient norm alone. Layerwise measurements are particularly useful because a global norm can hide which tensor is changing.

9.3 OLMo: stability involves data, architecture, and optimization

The OLMo 2 report [32] investigates stability through several ablations. Highly repeated short token patterns are one data-related source of spikes. The report also studies initialization, normalization placement, RMSNorm, query/key normalization, a logit-related auxiliary loss, and AdamW settings. The evidence supports a multi-cause view: filtering a pathological pattern can help without explaining every remaining incident.

Query/key normalization acts before the attention dot product. If Q and K grow in magnitude, the logits $QK^\top/\sqrt{d_h}$ can become extreme and the softmax highly concentrated. Controlling their scales can improve numerical and optimization behavior, but also changes the model's parameterization. It should be evaluated as an architectural choice, not inserted into a saved model without a recovery plan.

9.4 Marin: a recoverable spike and a damaged trajectory

The screenshots in the slides come from David Leo Wright Hall's report *Marin 32B Spike Fixing*. They distinguish frequent transient excursions from a smaller-looking spike that leaves the model on a worse trajectory. The height of the spike alone is therefore a poor severity measure.

The Marin 32B retrospective [18] documents instability around 70,000–80,000 steps. Several recovery attempts preceded a switch to a Qwen3-style backbone with query/key normalization, warm-started from an existing checkpoint. The report describes an initial disruption followed by recovery and improved stability. Even a large ongoing run can require a carefully tested architectural intervention.

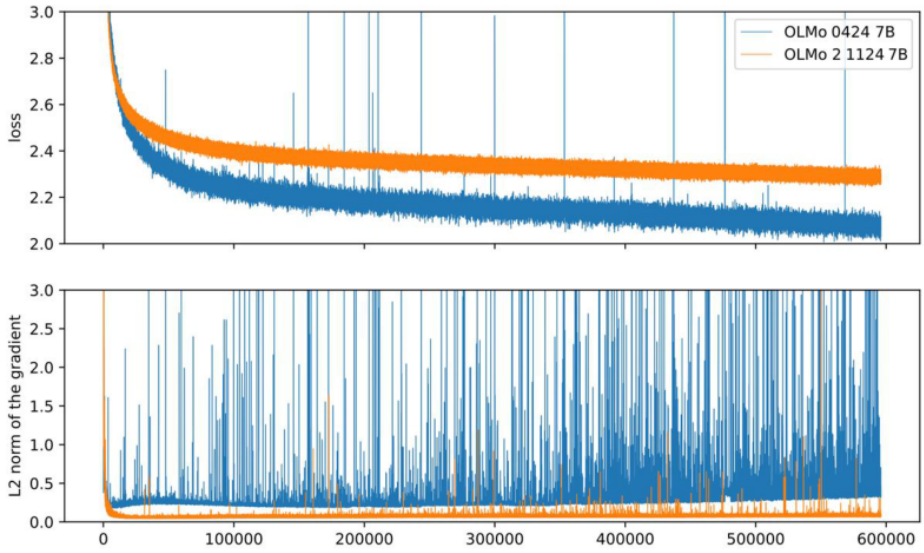


Figure 2: Training loss and gradient norm curves (over training steps) for OLMo-0424 and OLMo 2. The OLMo-0424 training run was characterized by frequent loss spikes (top), often preceded by more frequent spikes in the gradient norm, which grew over time (bottom). We note that overall training loss for OLMo 2 is higher because the underlying training data changed between the runs.

Figure 9.2: P2:21: OLMo versus OLMo 2. The newer run has far fewer loss and gradient spikes. Its higher absolute training loss cannot be read as worse quality because the underlying data distribution changed.

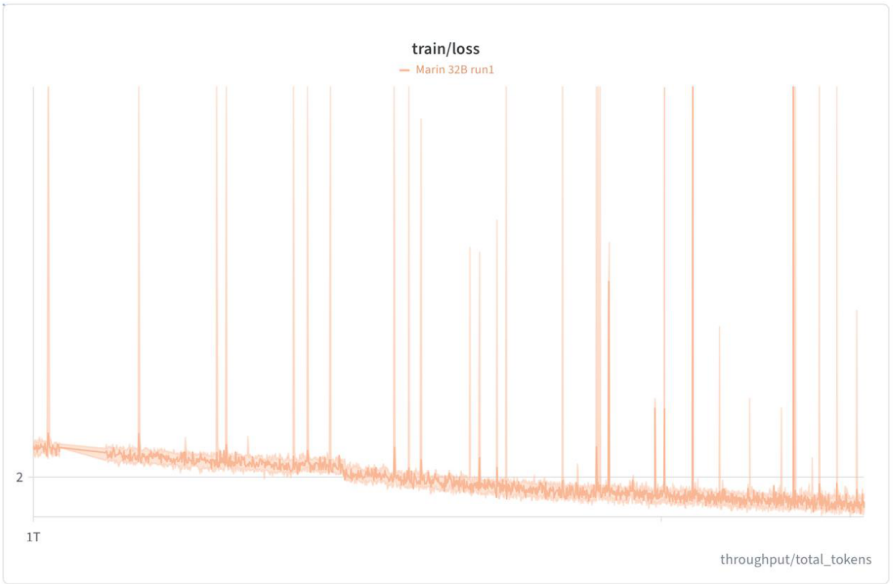


Figure 9.3: P2:22: the overall Marin loss trajectory, with many transient spikes. Continued improvement between them motivated investigating their lasting effect rather than stopping solely because a spike occurred.

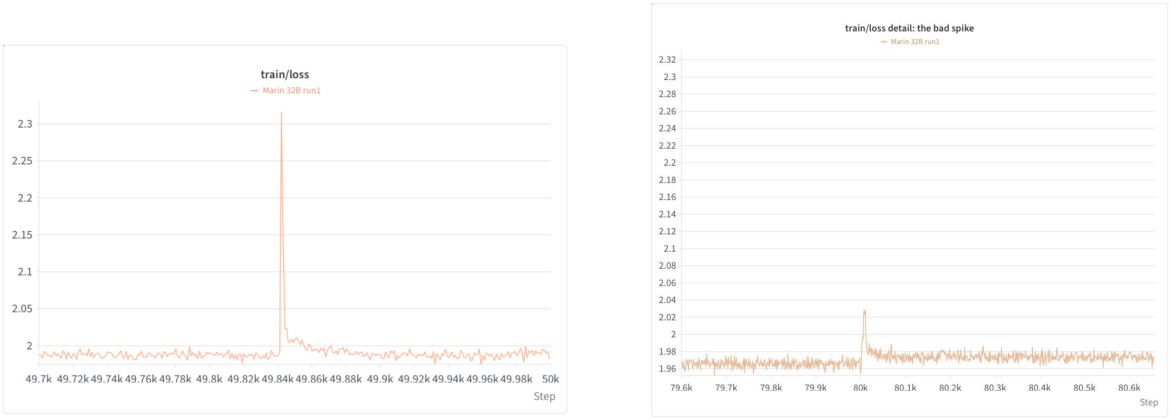


Figure 9.4: P2:22–23: a recoverable spike (left) and a persistent deterioration (right). The vertical ranges are similar; the worse incident need not have the largest instantaneous peak.

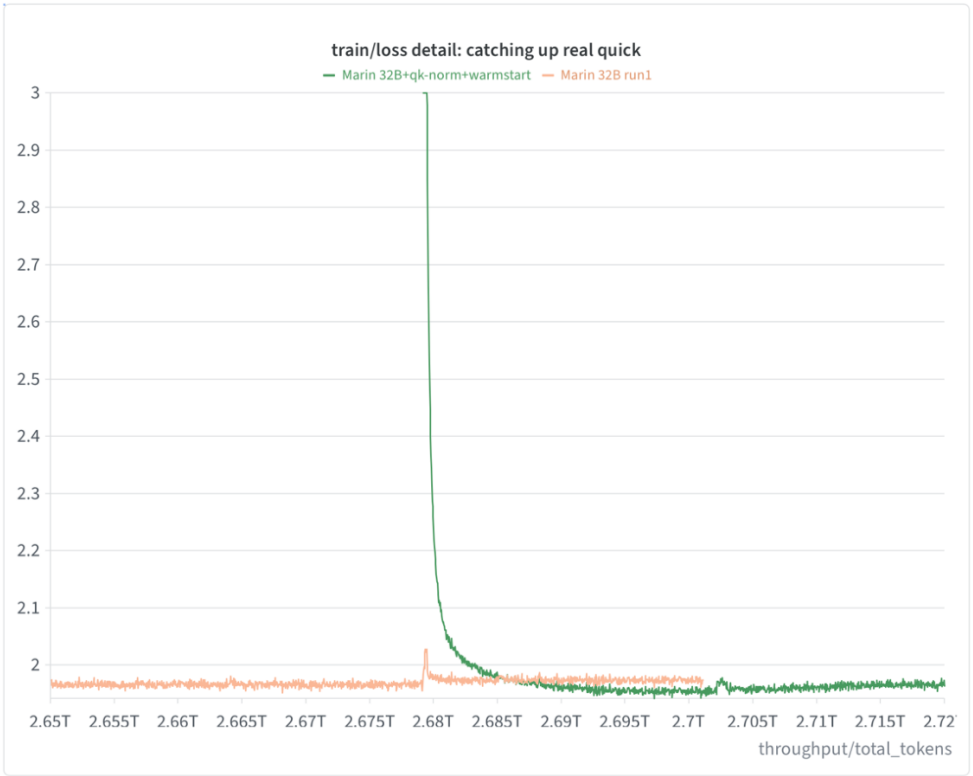


Figure 9.5: P2:23: the reported intervention and recovery. The original figure is retained as a case study, not a universal prescription for repairing all loss spikes.

9.5 A disciplined diagnostic procedure

First locate the onset precisely and preserve the last healthy checkpoint, its optimizer state, and its data position. Check whether the event coincides with a data-source change, unusually repetitive input, a schedule transition, a software change, or a hardware failure. Confirm that the metric itself has not changed.

Next compare short controlled continuations from the same checkpoint. One can replay the suspect batches, change their order, reduce the learning rate, inspect moment statistics, or test a targeted architecture or precision change. Change one factor at a time when feasible. A restart that happens to recover does not establish what caused the original failure.

Finally assess the recovered run on the same held-out data and against a healthy baseline. Do not judge recovery solely by returning to the numerical loss observed before a change in data mix. Document discarded compute and the exact continuation procedure so the final model's training history remains interpretable.

What to retain. Spikes are symptoms. Diagnose them using data provenance, optimizer dynamics, precision, and architecture together; a falling loss and finite numbers are necessary but insufficient evidence of a healthy run.

Chapter 10

Numerical precision, memory, and parallel training

Lecture slides: P2:24–33.

The previous chapters ask how much useful learning we obtain per token or FLOP. This chapter asks how quickly and reliably the hardware performs that work. A theoretically efficient recipe can lose in wall-clock time if it underuses the accelerator or communicates too much.

10.1 Floating-point formats: range and precision

A normal binary floating-point value has the form

$$x = (-1)^s 2^{e-b}(1.f),$$

where s is the sign, e the stored exponent, b its bias, and f the fraction. Exponent bits primarily determine dynamic range; fraction bits primarily determine relative precision. Special values and subnormals require additional conventions.

Format	Sign	Exponent	Fraction	Main tradeoff
FP32	1	8	23	Wide range and fine precision
FP16	1	5	10	Finer precision than BF16, narrower range
BF16	1	8	7	FP32-like exponent range, coarse precision
FP8 E4M3	1	4	3	More precision, less range than E5M2
FP8 E5M2	1	5	2	More range, less precision than E4M3
FP4 E2M1	1	2	1	Very coarse; scaling is essential

Table 10.1: Conceptual bit allocation. Exact encodings and supported operations depend on the format and hardware.

Near 1, the spacing between representable normal values is approximately 2^{-p} for p fraction bits. It is about 9.8×10^{-4} in FP16 and 7.8×10^{-3} in BF16. BF16 is less precise locally but is much less vulnerable to the narrow exponent range of FP16. This explains why range and precision must be discussed separately.

The small memory footprint of a tensor does not imply that every operation on it should use the same arithmetic. Matrix multiplications can use low-precision inputs with higher-precision accumulation. Reductions, optimizer states, normalization, and losses can retain more precision.

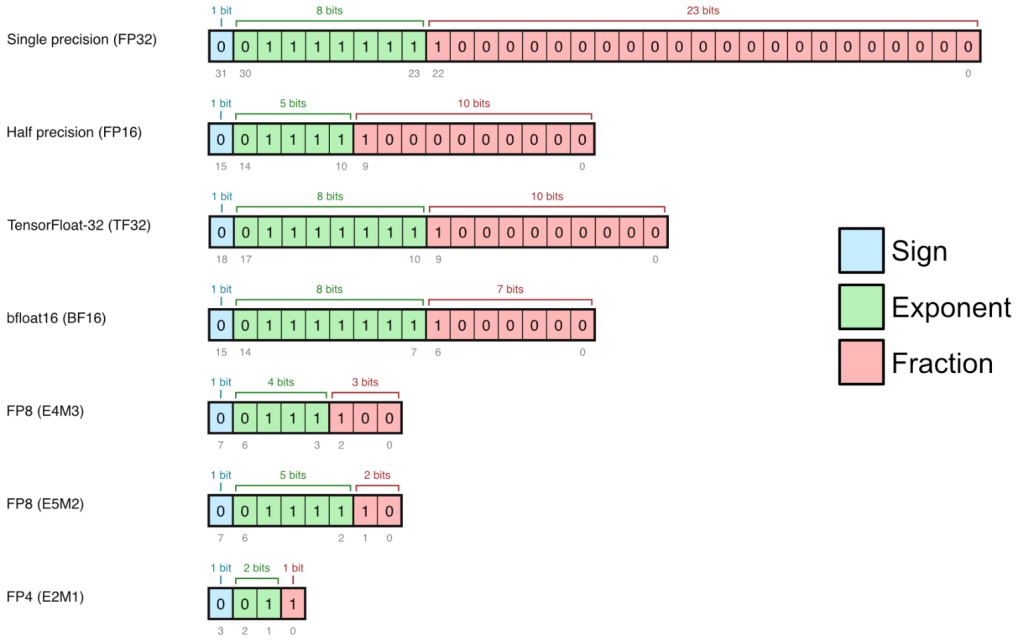


Figure 10.1: P2:25: bit allocations of common numerical formats. TF32 describes reduced-precision matrix arithmetic on compatible hardware; it is not an ordinary compact 19-bit tensor-storage type.

10.2 Mixed precision and why master weights matter

The mixed-precision training paper [20] combines low-precision arithmetic with safeguards such as higher-precision master weights, accumulation, and loss scaling. Consider a weight around 1 and a proposed update of 10^{-5}. Rounding the updated value immediately into BF16 can erase that update. Maintaining a more precise master copy lets small updates accumulate before casting a working copy for the next forward pass.

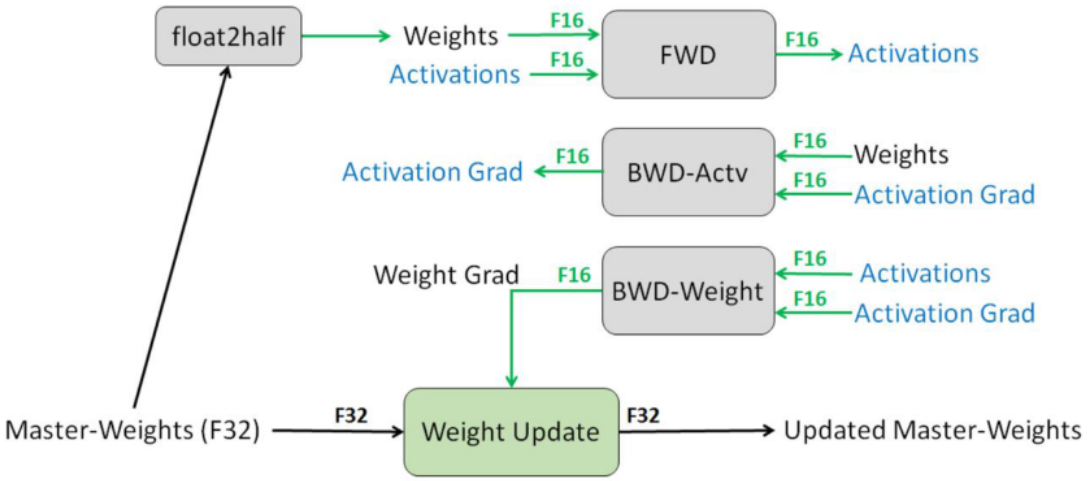


Figure 1: Mixed precision training iteration for a layer.

Figure 10.2: P2:26: mixed-precision data flow. Distinguish the precision of stored weights, matrix-multiplication inputs, accumulation, gradients, and optimizer states.

For FP16, multiplying the loss by a scale s also multiplies its gradients by s, helping small values remain representable. Gradients are divided by s before clipping and updating. Dynamic

scaling lowers s after overflow and skips invalid updates. BF16's wider exponent range often makes this particular mechanism unnecessary, although it does not eliminate rounding error or all possible overflows.

The "single PyTorch line" in P2:27 refers to autocasting eligible operations inside a context such as `torch.autocast`. It does not turn an arbitrary model and optimizer into a fully engineered low-precision training system. The chess loop also casts logits to float for the loss. Actual memory use depends on parameter storage, optimizer implementation, and temporary buffers; simply halving one tensor's bit width does not halve total training memory.

10.3 FP8: scale each tensor appropriately

FP8 has so few bits that dynamic range must be allocated carefully. A common conceptual operation is

$$q = \mathrm{round}_{\mathrm{FP8}}(x/s), \qquad \widetilde{x} = sq.$$

Choose s too large and small values lose precision; choose it too small and large values saturate. One scale for an entire tensor can be inefficient if a few outliers determine the range. Smaller groups can fit local ranges better, at the cost of scale metadata and more involved kernels.

The FP8 formats paper [21] discusses E4M3 and E5M2 as complementary precision/range choices. Their roles are implementation-dependent; "FP8 training" is not one universal format assignment.

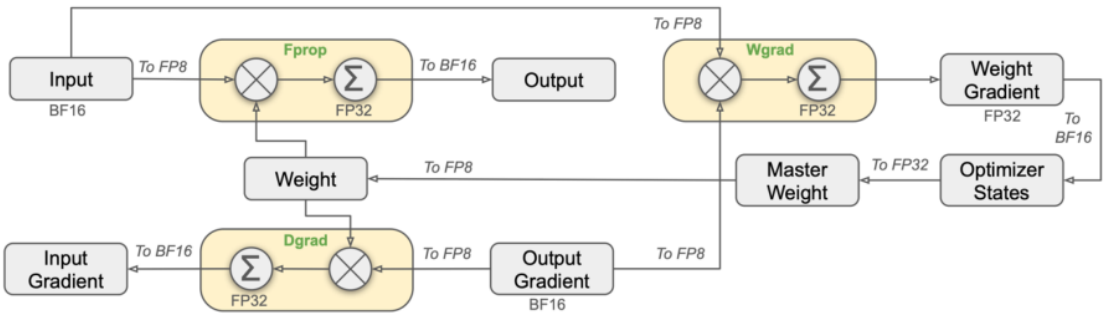


Figure 10.3: P2:28: the DeepSeek-V3 precision pipeline. Sensitive operations retain higher precision while expensive matrix multiplications use appropriately scaled FP8 inputs.

DeepSeek-V3 [9] uses fine-grained E4M3 quantization, including activation groups of 1×128 and weight blocks of 128×128, together with higher-precision accumulation and selected higher-precision operations. Its proposal is a coordinated numerical system. Copying only the low-precision dtype omits the mechanisms that make the recipe work.

Further explanation. A practical precision ablation should compare short runs from identical starting states, monitor layerwise overflow and update statistics, and evaluate loss differences at matched tokens. Faster matrix multiplication is useful only if the numerical changes do not require substantially more training or destabilize the run.

10.4 Activation recomputation

Backpropagation needs intermediate activations. Storing every layer's intermediates can dominate memory, especially for long sequences. Activation checkpointing stores selected boundaries, then

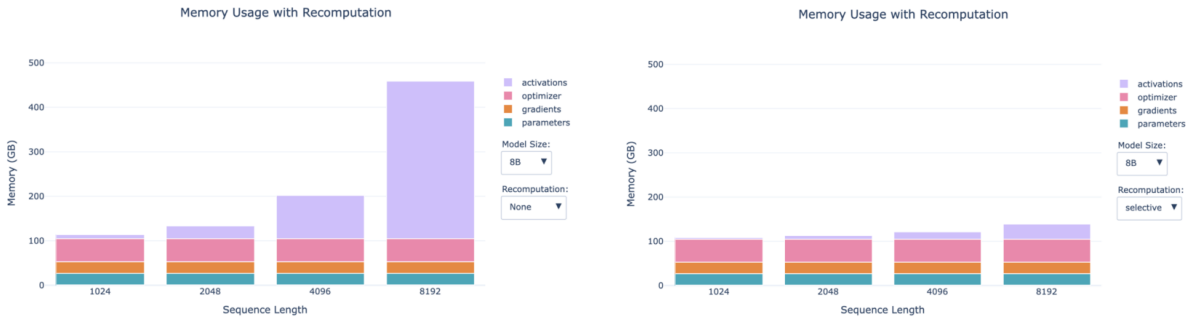


Figure 10.4: P2:30: the effect of recomputation for long sequences. The comparison illustrates the memory–compute tradeoff; the best operating point depends on the actual implementation and sequence length.

recomputes missing intermediate values during the backward pass. It exchanges extra computation for lower peak memory.

For a simple chain of L equally sized layers, save a boundary every k layers. The rough activation-storage requirement becomes proportional to

$$\frac{L}{k} + k.$$

The first term counts saved boundaries; the second counts a temporarily recomputed segment. Choosing $k \approx \sqrt{L}$ gives an $O(\sqrt{L})$ rather than $O(L)$ scaling in this simplified model. Chen et al. [5] develop sublinear-memory training strategies along these lines. Real transformer layers have heterogeneous intermediates, so the simple formula is an intuition, not an exact memory estimate.

Selective recomputation [15] avoids treating every operation equally. Recomputing a cheap elementwise operation may save considerable activation memory, whereas recomputing an expensive matrix multiplication may be unattractive. The cited work combines selective recomputation with sequence parallelism to reduce memory and redundant computation in large transformers.

Recomputation can improve *end-to-end* speed despite adding FLOPs if the saved memory enables a better microbatch, less offloading, or fewer communication stalls. It can also lower MFU when MFU counts only useful model FLOPs, even if the hardware is busy with recomputation. Always state the numerator of the utilization metric.

10.5 Three basic forms of parallelism

10.5.1 Data parallelism

Each worker holds a replica and processes a different portion of the batch. Gradients are combined before the update, typically by an all-reduce. The global batch is

$$B_{\text{global}} = B_{\text{micro}} \times n_{\text{accumulation}} \times n_{\text{replicas}}.$$

If valid-token counts differ between workers, aggregate loss sums and token counts consistently; averaging local means can otherwise weight workers incorrectly. Data parallelism increases throughput but does not by itself make one replica's model fit in memory. Sharding optimizer states, gradients, or parameters is an additional technique.

10.5.2 Pipeline parallelism

Assign consecutive groups of layers to different devices. Activations and their gradients cross stage boundaries. Multiple microbatches keep stages occupied, but startup and drain periods create pipeline bubbles. For an idealized balanced pipeline with p stages and m microbatches, a rough bubble fraction is $(p-1)/(m+p-1)$. Real schedules, imbalance, and communication alter this estimate.

10.5.3 Tensor parallelism

Partition a large layer's tensors across devices. For example, split output columns of a matrix multiplication and combine outputs when subsequent operations require them. This lets a single layer use multiple devices but introduces frequent communication inside the model. Fast interconnects and careful placement are crucial.

Method	What is partitioned?	Typical constraint
Data parallel	Batch examples	Gradient synchronization; replica memory
Pipeline parallel	Layer groups	Bubbles and stage imbalance
Tensor parallel	Operations within layers	Frequent high-bandwidth communication

Table 10.2: The three forms named in P2:31. Large systems combine them and may add sequence/context parallelism, expert parallelism, and state sharding.

The Ultra-Scale Playbook [12] develops these tradeoffs in practical detail. The right decomposition depends on the model, sequence length, batch size, available memory, and network topology. It is not determined by parameter count alone.

10.6 Mistral Large 4: a closing synthesis

Mistral Large 4 has 1 trillion total parameters and 52 billion active parameters. Its training used 3,840 GB200 GPUs, about 4.15 million GPU-hours over 45 days, and 36% MFU (P2:32). Active and total parameters play different roles for a mixture-of-experts model: compute follows the selected computation, while storage must accommodate the full model.

The technical report [23] confirms the hardware, duration, MFU, and AdamW betas $(0.975, 0.975)$. It specifies a 40M–80M–160M token batch schedule, whereas the slide abbreviates this as 50M→160M. The reported 65%/35% split denotes successive training phases, with the later phase emphasizing higher-quality and STEM data; it is not a two-source mixture ratio. The 28T-token total comes from P2:32 and is not independently confirmed here.

Using that slide-reported token count gives 28 tokens per *total* parameter but about 538 per *active* parameter. Neither ratio alone transfers the dense-model Chinchilla rule to a mixture-of-experts architecture. Its statistical capacity, active computation, routing, and full parameter storage need separate accounting.

The report predicts batch size and learning rate from scale-dependent variables and validates the recipe on a ladder of smaller runs. Its MFU uses peak BF16 throughput and excludes recomputation. Those conventions matter when comparing percentages across systems.

What to retain. Optimize useful learning per unit of wall-clock time. Numerical precision, memory savings, batch size, communication, and stability interact; an isolated improvement in FLOPs or throughput is not enough.

Chapter 11

Reproducing the experimental logic

11.1 The complete sequence

Separate experimental questions before combining them. The following sequence provides a practical plan for developing and validating a pretraining recipe.

1. **Establish a correct small baseline.** Verify causal masking, the one-token input/target shift, padding masks, and the held-out split. Check that the model can reduce loss on a tiny batch before a large sweep.
2. **Measure the hardware.** For a fixed model, vary microbatch size and record peak memory and tokens per second after warmup. Synchronize accelerator timing. Identify a useful throughput region and implement any required accumulation.
3. **Compare schedules at matched endpoints.** Run the cosine and WSD exercise with the same number of training examples. Branch a saved stable-phase state into shorter cooldowns and verify that optimizer state is restored.
4. **Locate the learning-rate basin.** Use a geometric sweep at 100,000 games. Inspect trajectories as well as final validation loss; repeat useful settings across seeds.
5. **Tune batch size jointly.** Retune the learning rate for each batch. Plot loss against games, updates, and elapsed time. Select a global batch that balances statistical and hardware efficiency.
6. **Fit horizon transfer.** Repeat learning-rate sweeps at several horizons. Fit local minima, then the log–log trend. Reserve at least one horizon to test the fitted rule.
7. **Validate width transfer.** Check parameter-group scaling and activation behavior, then verify alignment of base-learning-rate optima across widths.
8. **Construct isoFLOP curves.** At each budget, choose several model sizes and set their token horizons consistently. Ensure the observed minimum lies within the sweep.
9. **Forecast before scaling.** Fit the loss surface, record the next run's predicted loss and uncertainty, then perform the larger run. Report both successes and discrepancies.

11.2 What to save for every run

A useful experiment record contains the model configuration; parameter counts and their convention; tokenizer and vocabulary; dataset version, split, mixture and order; real and padded token counts; optimizer equations and parameter groups; global batch, microbatch and

accumulation; complete schedule; seed; precision; hardware; elapsed time; and code/configuration version. Save both final and best-checkpoint evaluation rules so that comparisons do not silently select different endpoints.

For a scaling fit, retain individual observations rather than only a plot. Record the uncertainty from seeds, the chosen fitting range, the loss metric, and whether coefficients use raw counts or millions. An apparent disagreement of several orders of magnitude can be a unit mismatch.

11.3 Worked checks

11.3.1 Token accounting and optimizer steps

At batch size 128 and padded length 256, a full batch contains 32,768 positions. If a game contributes 135 real tokens on average, it contains about 17,280 loss-bearing tokens instead. For 100,000 games there are approximately $100{,}000/128 = 781.25$ batches; the actual number of updates depends on whether the last partial batch is kept. There are 25.6 million padded positions and about 13.5 million real tokens. None of these quantities should be substituted for another without explanation.

11.3.2 Gradient accumulation

Four equal-sized microbatches of 32 games give a global batch of 128 on one device. Divide each microbatch loss by four before backpropagating if the microbatch losses are comparable means. If they contain unequal numbers of valid tokens, instead weight by valid-token counts so the accumulated gradient corresponds to the intended token-average objective. Update Adam's moments once per global batch, not after every microbatch.

11.3.3 Transferring the horizon rule

Using Equation (5.3), a horizon of 800,000 games gives

$$\eta^* = 0.0156 \times 8^{-0.36} \simeq 0.00738.$$

This is the fitted small-model rule. Width-dependent parameter groups still require their own scaling. The later operational normalization in Equation (7.5) would give a different numerical rate; choose the recipe deliberately.

11.3.4 Compute-optimal chess allocation

With the simplified rule $D = 65N$ and proxy budget $C = 10^{16}$,

$$N \simeq \sqrt{\frac{10^{16}}{6 \times 65}} \simeq 5.06 \text{ million}, \qquad D \simeq 329 \text{ million real tokens}.$$

At 135 real tokens per game, this is about 2.44 million games. It is close to, but not identical with, the separately fitted slide values because those use varying fitted ratios and rounded coefficients.

11.3.5 Interpreting a small loss improvement

For natural-log cross-entropy, perplexity is $\exp(\mathcal{L})$. A loss reduction from 0.916 to 0.904 changes perplexity by a multiplicative factor $e^{-0.012} \simeq 0.9881$, about a 1.2% reduction. This is a token-prediction measure; it does not directly translate into a percentage improvement in chess-playing strength or a downstream benchmark.

11.4 Questions for self-assessment

1. Why does $6ND$ omit some attention and padding costs? Which quantities would you measure to compare it with wall-clock time?
2. Why is L_2 regularization equivalent to weight decay for plain SGD but not for Adam?
3. Why does WSD allow one stable trajectory to serve several endpoints, and which state must be saved for the branches?
4. Why is an apparently optimal learning rate at one horizon insufficient for a scaling experiment?
5. What does the Chinchilla ratio optimize, and why might a deployed model rationally use many more tokens per parameter?
6. Why can a data filter lower training loss while reducing the usefulness of the final model?
7. What information would distinguish a harmless gradient-norm increase from an unstable parameter trajectory?
8. Under what circumstances could activation recomputation make a run faster despite adding computation?

Answers are developed in Chapters 2–10. The common thread is to specify what is fixed, what changes, and what quantity is being optimized before interpreting an experiment.

Appendix A

Editorial clarifications and figure provenance

This appendix collects clarifications needed to interpret the equations, experimental results, and cited sources consistently.

A.1 Corrections that affect interpretation

Tokens, games, and padded positions.
The throughput axis on P1:26 is in thousands of tokens per second, not games per second. The 100K-game experiment has about 13.5M real tokens and 25.6M padded positions. These quantities follow the axis labels and exercise specifications.

Adam's normalized update.
For a constant scalar gradient, $\sqrt{g^2} = |g|$, so the normalized direction is a sign, not always $+1$. The decay-as-EMA rewrite needs $u = -q/\lambda$ with the stated plus-sign interpolation. Chapter 3 supplies the corrected algebra.

Empirical fits are not universal constants.
P1:68 prints a width exponent about 1.58, whereas P1:70 shows about 1.34. The separate chess loss-surface exponents imply compute exponents different from the isoFLOP fit. Both discrepancies are retained and explained. A fit to a small number of runs should not be made artificially exact.

Absent image content.
The area below the formula on P1:88 is blank in the supplied file. The formula and its interpretation are included; no reconstructed plot is attributed to the slides.

Dates and scope of paper claims.
The large-batch paper was published in 2018. It includes language-model experiments. The model-collapse warning and the inverse-width intuition are stated with their assumptions rather than promoted to unrestricted theorems.

Mistral slide shorthand.
The report specifies 40M, 80M, and 160M token batches, compared with 50M→160M on the slide. Its 65%/35% refers to successive training phases. The 28T-token total remains attributed to P2:32. See Chapter 10 for the source comparison.

A.2 Figure provenance and consolidation

All figures labeled with P1 or P2 are crops of the supplied slide PDFs, preserving their plotted data and original labels. Their original research provenance is explained in the surrounding text and bibliography. The same chess anatomy figure appears in both decks and is reproduced once. Repeated sweep panels are consolidated; bibliographic title pages and implementation screenshots are represented by references or equations rather than reproduced as decorative images. The editable source package contains the figure manifest with source-page and crop coordinates.

For completeness, the additional seed-optimum panel and the alternate width fit are retained below. They support the uncertainty discussion rather than adding a new scaling claim.

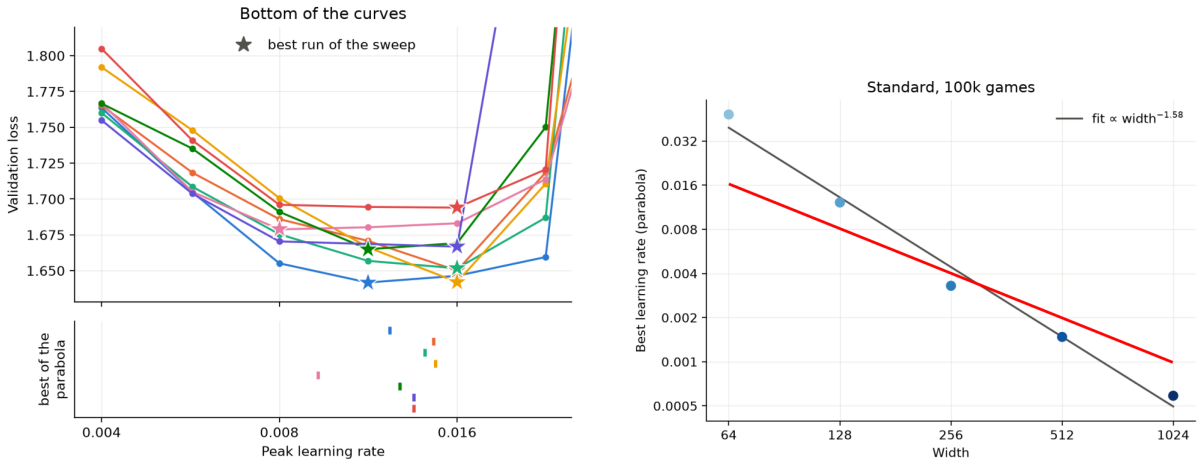


Figure A.1: Supplementary original panels from P1:51 and P1:70. Seed-dependent optimum estimates and an alternate width fit emphasize the limited precision of a small sweep.

Appendix B

A guide to the research and technical references

This guide summarizes the contributions of the cited references. The main chapters explain the mechanisms and experiments; the entries below suggest what to look for when reading the original source. References explicitly added to deepen the handout are marked as such. Blogs, software documentation, and technical reports are distinguished from research papers.

B.1 Optimization and hyperparameter transfer

Adam — Kingma and Ba [14]. The proposal is an adaptive optimizer combining first- and second-moment estimates with initialization-bias correction. Read its update equations beside Chapter 3. The useful question is how an apparently scalar learning rate interacts with coordinatewise preconditioning and the history of gradients.

AdamW — Loshchilov and Hutter [16]. The paper separates weight decay from the adaptive gradient transform. Its central distinction is algebraic: adding an L_2 penalty to Adam's gradient is not the same operation as shrinking parameters independently. This is the foundation for interpreting the AdamW decay coefficient.

Adaptive preconditioners and loss spikes — Bai et al. [1]. The proposal explains a class of spikes through mismatched second-moment dynamics and preconditioned curvature. Read the simplified stability analysis before the larger-network experiments. It supplies a mechanism to test, rather than a diagnosis that can be inferred from any jagged curve.

Why weight decay? — D'Angelo et al. [7]. The paper examines optimization-based explanations for decay beyond the conventional overfitting account. Compare its training regimes with nearly single-pass language modeling. Ask which assumptions would change when a curated source is repeated many times in a data mixture.

Scaling AdamW decay — Wang and Aitchison [33]. The proposal reparameterizes decay using an EMA timescale and investigates its transfer across model and dataset sizes. Re-derive the update with the correct sign in Chapter 3, then distinguish a timescale in updates from one in epochs. This is essential when batch size changes.

Late gradient growth — Defazio [10]. The paper links a late-training rise in gradient norm to decay and normalized parameters and proposes AdamC. Its relevance is diagnostic: raw gradient magnitude is not an invariant measure of the size of a functional model change. Check whether the mechanism's assumptions fit the layer under investigation.

Large-batch training — McCandlish et al. [19]. The proposal uses gradient noise to explain the diminishing benefit of larger batches. Compare the paper's data-efficiency and step-efficiency views with the paired chess plots in Chapter 5. A practical exercise is to estimate how the useful batch changes during a run rather than searching for one universal value.

Token-horizon transfer — Bjorck et al. [4]. The study fits how the best learning rate changes with the number of training tokens. Its contribution fills a gap left by model-size transfer. Read the measured variation between settings before turning the approximate exponent into a default recipe; the chess exponent is a separate empirical result.

Tensor Programs V and μP — Yang et al. [34]. The proposal makes width-dependent initialization and updates compatible with feature learning and hyperparameter transfer. The inverse-width argument in Chapter 6 is an entry point, not the full derivation. Use the reference implementation [22] to see how base shapes, readouts, initialization, and parameter groups fit together.

B.2 Scaling and large-run recipes

Kaplan scaling laws [13]. The paper establishes empirical regularities linking language-model loss to model size, data, and compute. Its allocation result motivates the later debate. Read its stopping and optimization conventions as carefully as the fitted exponents: a scaling curve includes the training procedure used to generate it.

Chinchilla — Hoffmann et al. [11]. The proposal allocates a fixed training budget more evenly between parameters and tokens. Compare its efficient-envelope, isoFLOP, and parametric-loss approaches. Chapter 7 derives why a balanced allocation arises when the two loss exponents are close, and why the token ratio is setting-dependent.

Resolving scaling discrepancies — Porian et al. [27]. The study revisits why Kaplan and Chinchilla disagree, controlling compute accounting and optimization details. Its results motivate tuning before scaling. A poor small-run recipe can produce a systematic, rather than random, forecasting error.

DeepSeek LLM [8]. This technical report connects scaling experiments to a complete model-training recipe. It includes alternative learning-rate schedules and empirical batch/rate scaling. Read the units and fixed settings attached to each fitted expression before attempting to reuse its coefficients.

Marin's extrapolation report [17]. This is a primary engineering/research blog, not a conventional conference paper. It illustrates making predictions beyond the fitted compute range. The useful comparison with the chess experiment is the separation between fitting, committing to a forecast, and evaluating the later run.

Mistral Large 4 technical report [23]. This report provides an industrial example of pretraining at scale. Read the scaling-ladder and training sections together: the hardware-efficient implementation and the transferred statistical recipe are jointly responsible for the result. Chapter 10 records the differences between the slide shorthand and the report.

B.3 Data sources, curation, and mixtures

FineWeb and FineWeb-Edu — Penedo et al. [26]. The core proposal is an empirically tested web-data pipeline. Read its ablations alongside the processing diagram in Chapter 8, and ask what each stage changes in the remaining distribution. The educational subset illustrates model-assisted selection rather than simple URL-based filtering.

Common Crawl [6]. This is infrastructure documentation, not a modeling paper. Use it to understand the archive representations and their intended contents. The WARC/WET comparison in Chapter 8 shows why selecting a download format is already a consequential extraction choice.

Trafilatura — Barbaresi [2]. This system-demonstration paper presents a tool for discovering and extracting web text. The tool converts raw page archives into useful content. Evaluate extraction on representative pages from the intended domains, including failure cases.

Gopher — Rae et al. [28]. The report provides the data-quality heuristics used in Chapter 8. Read those heuristics as operational choices in a specific pipeline, not as general definitions of valuable language. They provide a useful baseline against which model-based filtering can be tested.

FinePhrase — Niklaus et al. [24]. The proposal systematically compares synthetic rephrasing choices and releases the resulting dataset. The questions to carry into a reproduction are whether the chosen output format preserves information, whether generator cost is justified, and whether gains survive a change in the real-data mixture.

Nemotron-CC — Su et al. [31]. The report combines filtering and synthetic transformation for long-horizon pretraining. It is a complementary example to FineWeb: alternative choices about selection, retention, and rewriting can be tested by training downstream models rather than by inspecting text alone.

OLMo 2 [32]. This open-model report supplies the stability comparison in Chapter 9 as well as a broader training recipe. The pretraining-stability section is the most relevant reading here. Its multiple ablations are a useful antidote to searching for one explanation for every loss spike.

Olmo 3 [25]. The report details pretraining, midtraining, and long-context mixtures. Read the tables by separating available source pools from sampled training tokens. The key question is how a stage's objective and source availability determine the chosen proportions and repetition rates.

Model collapse — Shumailov et al. [30] (additional reading). This reference analyzes the risk of model collapse from recursively generated data. Read the data-replacement assumptions

carefully. Its setting should be distinguished from a single round of source-grounded rewriting mixed with retained real data, as discussed in Chapter 8.

B.4 Numerics, memory, and systems

Mixed precision — Micikevicius et al. [20]. The proposal uses low-precision computation while retaining selected higher-precision state and safeguards. Track the dtype of each object through the diagram in Chapter 10. "The model uses FP16" is too imprecise to specify a training implementation.

FP8 formats — Micikevicius et al. [21]. The paper proposes complementary eight-bit formats and studies their use in deep learning. The relevant distinction is dynamic range versus precision. Understand both before interpreting a claim that one format is better for a given tensor.

DeepSeek-V3 [9]. The FP8 training system is the relevant contribution for Chapter 10. Read the quantization-group and accumulation choices together with the higher-precision exceptions. The numerical design is more informative than the headline bit width.

Sublinear-memory training — Chen et al. [5]. The proposal trades recomputation for activation storage. Chapter 10 derives the simple checkpoint-spacing intuition. Compare that chain model with the heterogeneous cost of attention, matrix multiplications, and elementwise operations in a transformer.

Selective recomputation — Korthikanti et al. [15]. The paper combines selective activation recomputation with sequence parallelism. Its useful systems principle is to avoid recomputing expensive intermediates indiscriminately. Evaluate both memory savings and the resulting throughput at the batch size those savings permit.

Marin 32B retrospective [18]. This primary engineering report documents a run's incidents and interventions. The value is the causal sequence and the failed recovery attempts, not just the successful final configuration. It complements the more controlled stability analyses.

The Smol Training Playbook [3]. This practitioner reference organizes the end-to-end process of developing a training recipe. It is useful for placing small, controlled experiments in a larger model-development workflow.

The Ultra-Scale Playbook [12]. This systems guide develops the memory, communication, and parallelism tradeoffs introduced in Chapter 10. Use it after the three-way comparison in Chapter 10 to study how those forms of parallelism are combined on real clusters.

PyTorch GPU-memory visualization [29]. This primary software article explains inspecting allocations over time. It is the practical continuation of Exercise 1: identify what actually occupies memory rather than inferring the peak solely from parameter count.

Bibliography

[1] Zhiwei Bai et al. Adaptive preconditioners trigger loss spikes in adam. *arXiv:2506.04805*, 2025. URL https://arxiv.org/abs/2506.04805.

[2] Adrien Barbaresi. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *ACL System Demonstrations*, pages 122–131, 2021. URL https://aclanthology.org/2021.acl-demo.15/.

[3] Loubna Ben Allal et al. The smol training playbook: The secrets to building world-class llms. Hugging Face, 2025. URL https://huggingfacetb-smol-training-playbook.hf.space/.

[4] Johan Bjorck, Alon Benhaim, Vishrav Chaudhary, Furu Wei, and Xia Song. Scaling optimal lr across token horizons. *ICLR*, 2025. URL https://arxiv.org/abs/2409.19913.

[5] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv:1604.06174*, 2016. URL https://arxiv.org/abs/1604.06174.

[6] Common Crawl. Get started: Crawl archives and data formats, 2026. URL https://commoncrawl.org/get-started.

[7] Francesco D'Angelo, Maksym Andriushchenko, Aditya Varre, and Nicolas Flammarion. Why do we need weight decay in modern deep learning? *NeurIPS*, 2024. URL https://arxiv.org/abs/2310.04415.

[8] DeepSeek-AI et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv:2401.02954*, 2024. URL https://arxiv.org/abs/2401.02954.

[9] DeepSeek-AI et al. Deepseek-v3 technical report. *arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

[10] Aaron Defazio. Why gradients rapidly increase near the end of training. *arXiv:2506.02285*, 2025. URL https://arxiv.org/abs/2506.02285.

[11] Jordan Hoffmann et al. Training compute-optimal large language models. *NeurIPS*, 2022. URL https://arxiv.org/abs/2203.15556.

[12] Hugging Face Nanotron Team. The ultra-scale playbook: Training llms on gpu clusters, 2025. URL https://nanotron-ultrascale-playbook.static.hf.space/index.html.

[13] Jared Kaplan et al. Scaling laws for neural language models. *arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

[14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *ICLR*, 2015. URL https://arxiv.org/abs/1412.6980.

[15] Vijay Korthikanti et al. Reducing activation recomputation in large transformer models. *arXiv:2205.05198*, 2022. URL https://arxiv.org/abs/2205.05198.

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2019. URL https://arxiv.org/abs/1711.05101.

[17] Marin Community. Scaling laws that extrapolate 300 times past the fit, 2026. URL https://marin.community/blog/2026/05/11/delphi/.

[18] Marin Community. Marin 32b retrospective, n.d. URL https://github.com/marin-community/marin/blob/main/docs/reports/marin-32b-retro.md. Accessed 6 November 2026.

[19] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv:1812.06162*, 2018. URL https://arxiv.org/abs/1812.06162.

[20] Paulius Micikevicius et al. Mixed precision training. *ICLR*, 2018. URL https://arxiv.org/abs/1710.03740.

[21] Paulius Micikevicius et al. Fp8 formats for deep learning. *arXiv:2209.05433*, 2022. URL https://arxiv.org/abs/2209.05433.

[22] Microsoft Research. Maximal update parametrization: Reference implementation, 2022. URL https://github.com/microsoft/mup.

[23] Mistral AI. Mistral large 4. Technical report, Mistral AI, 2026. URL https://huggingface.co/mistralai/Mistral-Large-4-1T-A52B/resolve/main/assets/technical_report.pdf.

[24] Joel Niklaus et al. How can we synthesize high-quality pretraining data? a systematic study of prompt design, generator model, and source data. *arXiv:2604.13977*, 2026. URL https://arxiv.org/abs/2604.13977.

[25] Olmo Team et al. Olmo 3. *arXiv:2512.13961*, 2025. URL https://arxiv.org/abs/2512.13961. Revised April 2026.

[26] Guilherme Penedo et al. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv:2406.17557*, 2024. URL https://arxiv.org/abs/2406.17557.

[27] Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. *NeurIPS*, 2024. URL https://arxiv.org/abs/2406.19146.

[28] Jack W. Rae et al. Scaling language models: Methods, analysis and insights from training gopher. *arXiv:2112.11446*, 2021. URL https://arxiv.org/abs/2112.11446.

[29] Aaron Shi and Zachary DeVito. Understanding gpu memory 1: Visualizing all allocations over time. PyTorch, 2023. URL https://pytorch.org/blog/understanding-gpu-memory-1/.

[30] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *arXiv:2305.17493*, 2023. URL https://arxiv.org/abs/2305.17493. Revised April 2024; additional reading.

[31] Dan Su et al. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset. *arXiv:2412.02595*, 2024. URL https://arxiv.org/abs/2412.02595.

[32] Team OLMo et al. 2 olmo 2 furious. *arXiv:2501.00656*, 2025. URL https://arxiv.org/abs/2501.00656.

[33] Xi Wang and Laurence Aitchison. How to set adamw's weight decay as you scale model and dataset size. *ICML*, 2025. URL https://arxiv.org/abs/2405.13698.

[34] Greg Yang et al. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv:2203.03466*, 2022. URL https://arxiv.org/abs/2203.03466.